

ISO/IEC 27001 ומשפחת תקני ISO/IEC

27000 – ספר סיכום

ספר סיכום מקיף בעברית על תקן **ISO/IEC 27001** (ניהול אבטחת מידע), על מבנהו, נספח הבקורות שלו, תהליך ההסמכה, ועל **התקנים הנגזרים** ממשפחת ISO/IEC 27000 – כולל תקנים ייעודיים לענן, לפרטיות, לאבטחת סייבר, לניהול אירועים ועוד.

הספר נכתב כחומר לימוד/עיון למקצועני אבטחת מידע, מנהלי סיכונים, מבקרים פנימיים, ואנשי מקצוע המובילים תהליכי הטמעה או הסמכה בארגון.

הבהרה: מסמך זה הוא סיכום לימודי ואינו תחליף לרכישת נוסח התקן הרשמי מ-ISO/IEC, ואינו ייעוץ משפטי או רגולטורי. לפני הטמעה או הסמכה בפועל יש להיעזר בנוסח התקן המלא ובגורם מקצועי מוסמך (יועץ/מבקר/גוף הסמכה מוכר).

גרסת PDF: כל הספר (כל הפרקים + נספח הכלים) מאוחד גם לקובץ יחיד – **ISO-27001-Summary-Book-HE.pdf**. הפקה מחדש לאחר עדכון פרק: `python3 tools/build_book_pdf.py` (ראו tools/README.md).

תוכן העניינים

#	פרק	תיאור קצר
1	מבוא ומשפחת התקנים	מהי אבטחת מידע, מהו ISMS, רקע היסטורי, מבנה משפחת ISO/IEC 27000
2	מבנה התקן ISO/IEC 27001	סעיפים 4-10, מבנה PDCA, Annex SL, הקשר ארגוני, מנהיגות, תכנון, תמיכה, תפעול, הערכה, שיפור
3	נספח A – בקורות אבטחת מידע	93 הבקורות (מהדורת 2022) ב-4 קטגוריות: ארגוניות, אנשים, פיזיות, טכנולוגיות
4	תקנים נגזרים ממשפחת ISO/IEC 27000	27002, 27005, 27017, 27018, 27701, 27032, 27035, 27036, 22301 ועוד
5	תהליך ההסמכה והביקורת	שלבי הסמכה, ביקורת Stage 1/Stage 2, ביקורות מעקב, גופי הסמכה
6	יישום מעשי בארגון	מפת דרכים להטמעה, הערכת סיכונים, SoA, מדדים, תרבות אבטחה
7	2013 מול 2022 – מה השתנה	ההבדלים המרכזיים בין המהדורות, השלכות על ארגונים מוסמכים
8	מילון מונחים ורשימת בדיקה	מונחי מפתח + Checklist מעשי להטמעה/הסמכה

#	פרק	תיאור קצר
9	בדיקות חדירה (Pentesting) ו-ISO/ IEC 27001	מיפוי pentest לבקורות נספח A, סוגי בדיקות ומתודולוגיות (כולל CompTIA ROE, PenTest+ ואתיקה, דוגמה עובדת שממירה ממצאי pentest אמיתיים לרשומת סיכונים, CVSS ו-OWASP Top 10, תקינה ורגולציה בינלאומית (CREST/TIBER-EU/DORA/PCI DSS) והתאמה למציאות הישראלית
10	פרוטוקולי בדיקה מפורטים לפי CWE	לכל אחד מ-41 ה-CWE בכלי הנלווה: מטרה, תנאים מוקדמים, שלבי בדיקה, קריטריון הצלחה וכלים לדוגמה
11	כיצד מתגלים חולשות Zero-Day	מתודולוגיית מחקר חולשות (fuzzing, סקירת קוד, הנדסה לאחור, patch diffing), גילוי אחראי (ISO/IEC 29147/30111), והשלכות על הגנת-עומק במסגרת ISMS

כלי נלווה: מחשבון סקר סיכונים לפי CWE

בתיקית [tools/](#) נמצא כלי שורת-פקודה אופליין (`cwe_risk_calculator.py`) שממחיש כיצד הופכים ממצאי חולשות טכניות (ממופות למזי **CWE**) לרשומת סיכונים מדורגת, לפי מודל Likelihood × Impact בהתאם למתודולוגיית ISO/IEC 27005 — ראו התייחסות מפורטת בפרק 6 (6.3) ודוגמה מבוססת-pentest אמיתי בפרק 9 (9.6).

איך לקרוא את הספר

- אם אתם חדשים לנושא — התחילו מפרק 1 והתקדמו לפי הסדר.
- אם אתם כבר מכירים את התקן ומחפשים רענון על הבקורות — קפצו ישירות לפרק 3.
- אם אתכם מעניינים תקנים ספציפיים (ענן, פרטיות, המשכיות עסקית) — פרק 4.
- אם אתם מתכוננים להסמכה בפועל — פרקים 5-6 ו-8 (ה-Checklist) הם החומר המעשי.

פרק 1: מבוא ומשפחת התקנים

1.1 מהי אבטחת מידע ומהי ISMS

אבטחת מידע (Information Security) עוסקת בהגנה על מידע מפני איומים, כדי להבטיח שלושה מאפיינים מרכזיים — המכונים **משולש ה-CIA**:

- **Confidentiality (סודיות)** — המידע נגיש רק למי שמורשה לגשת אליו.
- **Integrity (שלמות)** — המידע מדויק, שלם, ולא שונה ללא הרשאה.
- **Availability (זמינות)** — המידע והמערכות זמינים למי שזקוק להם, כשהוא זקוק להם.

תקנים מודרניים (כולל ISO/IEC 27001:2022) מוסיפים לעיתים גם מאפיינים כמו **אותנטיות (Authenticity)**, **אחריות (Accountability)**, **אי-התכחשות (Non-repudiation)** ו**אמינות (Reliability)**.

ISMS — Information Security Management System (מערכת ניהול אבטחת מידע)

היא לא "מוצר" או "מערכת טכנולוגית" אחת, אלא **מערכת ניהולית**: מדיניות, תהליכים, נהלים, תפקידים, אחריות, ומשאבים שהארגון מפעיל באופן שיטתי ומתמשך כדי לנהל את הסיכונים לאבטחת המידע שלו. זהו ההבדל המרכזי בין "יש לנו פיירוול וסיסמאות חזקות" לבין "יש לנו ISMS" — הראשון הוא אוסף בקורות נקודתיות, השני הוא מערכת ניהול שיטתית שמתעדכנת, נמדדת, ומשתפרת לאורך זמן.

1.2 רקע היסטורי

- **BS 7799 (1995)** — תקן בריטי (BSI) שהיה הבסיס למה שיהפוך מאוחר יותר לתקן הבינלאומי. חלק 1 (Code of Practice) וחלק 2 (Specification for ISMS).
- **ISO/IEC 17799 (2000)** — אימוץ בינלאומי של BS 7799 Part 1 כקוד התנהגות לבקורות אבטחת מידע.
- **BS 7799-2:2002** — עדכון חלק 2 (דרישות ל-ISMS) שכלל את מודל ה-PDCA (Plan-Do-Check-Act) של דמינג.
- **ISO/IEC 27001:2005** — הפרסום הבינלאומי הראשון של תקן הדרישות עצמו, מבוסס על BS 7799-2:2002. זהו ה"לידה" הרשמית של משפחת 27000.
- **ISO/IEC 27002:2005** — שינוי שם ל-ISO/IEC 17799:2005 (קוד ההתנהגות לבקורות), שקיבל מספור מחדש כ-27002 ב-2007 כדי להתאים למשפחה.
- **ISO/IEC 27001:2013** — עדכון מהותי: מעבר למבנה האחיד **Annex SL** המשותף לכל תקני מערכות הניהול של ISO (למשל ISO 9001, ISO 14001), עדכון נספח A ל-114 בקורות ב-14 קטגוריות (domains).
- **ISO/IEC 27001:2022** — המהדורה הנוכחית. הדרישות עצמן (סעיפים 4-10) עודכנו בעדכונים נקודתיים, אך השינוי הבולט ביותר הוא **ריענון מלא של נספח A**: מ-114 בקורות ב-14 קטגוריות

1.3 מבנה משפחת ISO/IEC 27000

משפחת ISO/IEC 27000 היא אוסף תקנים שכולם עוסקים בהיבטים שונים של ניהול אבטחת מידע, ובנויים סביב "גרעין" משותף — **ISO/IEC 27001** (הדרישות שאפשר להיסמך עליהן) ו-**ISO/IEC 27002** (קטלוג הבקורות המפורט). שאר התקנים במשפחה הם:

- **תקני תמיכה כלליים** — מדריכי יישום, מדידה, ביקורת, ניהול סיכונים.
- **תקני מגזר** — התאמות התקן לתעשיות ספציפיות (בריאות, אנרגיה, טלקום, ענן).
- **תקני נושא** — העמקה בנושא ספציפי (פרטיות, סייבר, פורנזיקה דיגיטלית, ניהול אירועים, יחסי ספקים).

חשוב להבין: רק **ISO/IEC 27001** הוא תקן "הניתן להסמכה" (**certifiable**) — כלומר ארגון יכול לעבור ביקורת של גוף הסמכה חיצוני ולקבל תעודת התאמה רשמית לתקן זה. כל שאר התקנים במשפחה הם **תקני הנחיה (guidance)** — הם עוזרים ליישם, להעמיק, או להרחיב את ה-ISMS, אך אין "תעודת ISO 27002" או "תעודת ISO 27005" במובן הפורמלי (למעט חריגים כמו ISO/IEC 27701, שניתן להסמיק אליו כתוספת ל-27001, וכן PIMS/PIPS extension). פירוט מלא של התקנים הנגזרים נמצא בפרק 4.

1.4 מודל Annex SL ו-PDCA

מאז מהדורת 2013, ISO/IEC 27001 בנוי לפי **Annex SL** (High Level Structure) — מבנה אחיד עם 10 סעיפים המשותף לכל תקני מערכות הניהול של ISO. המשמעות המעשית: ארגון שכבר מיישם ISO 9001 (איכות) או ISO 14001 (סביבה) ימצא מבנה מוכר ויוכל לשלב (Integrated Management System) בין המערכות בקלות יחסית.

המודל התפעולי הבסיסי הוא **PDCA — Plan, Do, Check, Act**:

1. **Plan (תכנון)** — הגדרת היקף ה-ISMS, מדיניות, הערכת סיכונים, תכנית טיפול בסיכונים.
2. **Do (ביצוע)** — יישום הבקורות והתהליכים שהוחלט עליהם.
3. **Check (בדיקה)** — ניטור, מדידה, ביקורות פנימיות, סקר הנהלה.
4. **Act (שיפור)** — טיפול באי-התאמות, פעולות מתקנות, שיפור מתמשך.

מחזור זה **אינו חד-פעמי** — הוא לב ליבו של הרעיון ש-ISMS הוא מערכת חיה ומתמשכת, ולא פרויקט עם קו סיום.

1.5 מדוע ארגונים בכלל מיישמים את התקן

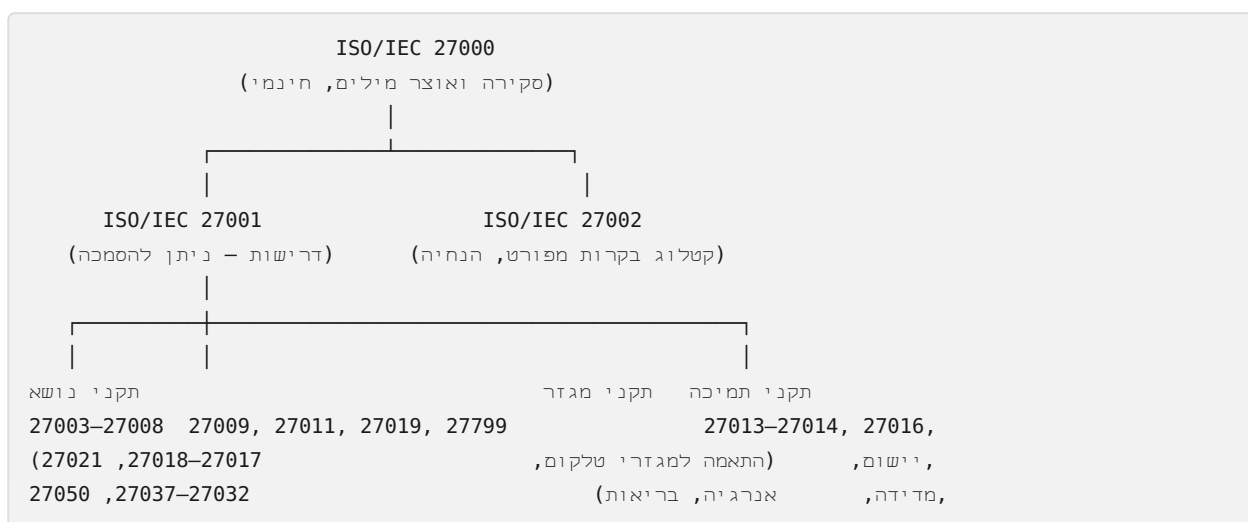
לפני שנכנסים לפרטים הטכניים (פרקים 2-3), שווה לעצור על השאלה שכל ISMS אמיתי צריך לענות עליה מההתחלה: **מה בשביל מה?** הסיבות הנפוצות ביותר בפועל, ולרוב כמה מהן יחד:

- **דרישת לקוחות/מכרזים** — הסיבה השכיחה ביותר בפועל. לקוחות ארגוניים (B2B, במיוחד ספקי SaaS/עיצוב מידע) דורשים הוכחת הסמכה כתנאי סף לחתימת חוזה או להשתתפות במכרז.
- **דרישה רגולטורית/חוזית עקיפה** — רגולציות כמו GDPR, NIS2 (אירופה), או חוק הגנת הפרטיות הישראלי לא דורשות ISO 27001 במפורש, אך הסמכה מספקת "עדות מוחשית" נוחה להוכחת נאותות אמצעי הגנה (Adequate Security Measures) מול רגולטור או מול לקוח.
- **יתרון תחרותי ואמון שוק** — תעודה מוכרת בינלאומית מקצרת משא ומתן על שאלוני אבטחת ספקים (Vendor Security Questionnaires) ומאיצה מכירות.
- **הפחתת סיכון אמיתי** — כשהיישום נעשה ברצינות (לא רק "לצורך התעודה"), ISMS שיטתי מפחית בפועל את הסתברות והיקף אירועי אבטחת מידע.
- **ביטוח סייבר (Cyber Insurance)** — מבטחים רבים משפרים תנאי פוליסה (פרמיה, כיסוי, השתתפות עצמית) לארגונים עם ISMS מוסמך, כחלק מהערכת הסיכון החיתומית שלהם.
- **בדיקת נאותות (Due Diligence) ברכישות/מיזוגים** — הסמכה קיימת מקלה משמעותית על תהליכי בדיקת נאותות אבטחת מידע לפני עסקה.
- **ממשל תאגידי ואחריות דירקטוריון** — ISMS מובנה נותן לדירקטוריון/הנהלה מנגנון דיווח ובקרה סדור על סיכוני אבטחת מידע, ולא תלות באדם בודד.

אזהרה חשובה: ארגון שמיישם ISO 27001 **רק** כדי "לסמן וי" מול לקוח, בלי מחויבות ניהולית אמיתית (סעיף 5.1), בדרך כלל מגיע לביקורת עם ISMS שברירי — ולעיתים אף נכשל בביקורת ההסמכה עצמה. ISMS ש"עובד על הנייר" אך לא הופעל בפועל הוא לרוב יקר יותר (בזמן ובכסף) מהטמעה כנה מלכתחילה.

1.6 מפת משפחת התקנים — תרשים

התרשים הבא (מפורט בפרק 4) ממחיש את המבנה ההיררכי של המשפחה: גרעין הניתן-להסמכה, ומעליו שכבות הנחיה שמעשירות אותו:



הרעיון המרכזי: ככל שיורדים במפה, התקנים נעשים ממוקדים וספציפיים יותר — אך כולם "נשענים" בסופו של דבר על אותו גרעין דרישות (27001) וקטלוג בקורות (27002). ארגון לא צריך "לדעת" את כל +40 התקנים במשפחה — מספיק להכיר את אלו הרלוונטיים להקשר שלו (ראו טבלת הסיכום בפרק 4, סעיף 4.6).

פרק 2: מבנה התקן ISO/IEC 27001

תקן ISO/IEC 27001 מחולק לשני חלקים עיקריים:

1. **הגוף הראשי (סעיפים 4-10)** — הדרישות שארגון **חייב** לעמוד בהן כדי להיות מוסמך. אלו דרישות "לא ניתנות למיקוח" (must), בשונה מהבקורות בנספח A שניתן להחריג בהתאם להערכת סיכונים ול-SoA.
2. **נספח A** — קטלוג בקורות אבטחת מידע (מפורט בפרק 3), שמשמש כ"רשימת קניות" שממנה הארגון בוחר אילו בקורות רלוונטיות לו, בהתבסס על תוצאות הערכת הסיכונים.

סעיף 4 — הקשר הארגוני (Context of the Organization)

- **4.1 הבנת הארגון והקשרו** — זיהוי סוגיות פנימיות וחיצוניות הרלוונטיות למטרת הארגון ולמסוגלותו להשיג את התוצאות הצפויות מה-ISMS (למשל: רגולציה, תחרות, טכנולוגיה, תרבות ארגונית).
- **4.2 הבנת הצרכים והציפיות של בעלי עניין** — לקוחות, רגולטורים, עובדים, ספקים, בעלי מניות — ואילו מהדרישות שלהם רלוונטיות ל-ISMS (כולל דרישות חוזיות ורגולטוריות כמו GDPR, חוק הגנת הפרטיות הישראלי, PCI DSS וכו').
- **4.3 קביעת היקף ה-ISMS (Scope)** — הגדרה מדויקת אילו יחידות ארגוניות, מיקומים, נכסים וטכנולוגיות נכללים ב-ISMS. ההיקף חייב להיות מתועד וזמין כמידע מתועד. **זהו אחד הסעיפים הקריטיים ביותר** — היקף מוגדר בצורה רופפת או רחבה מדי מסבך את ההטמעה וההסמכה.
- **4.4 מערכת ניהול אבטחת המידע** — דרישה כללית להקים, ליישם, לתחזק ולשפר באופן מתמשך ISMS בהתאם לדרישות התקן.

סעיף 5 — מנהיגות (Leadership)

- **5.1 מנהיגות ומחויבות** — ההנהלה הבכירה (Top Management) חייבת להפגין מחויבות פעילה: הבטחת התאמת מדיניות אבטחת המידע לכיוון האסטרטגי, הקצאת משאבים, קידום שיפור מתמשך, ותמיכה בתפקידים רלוונטיים.
- **5.2 מדיניות** — קביעת מדיניות אבטחת מידע מתועדת, מותאמת למטרת הארגון, כוללת מחויבות לעמידה בדרישות ולשיפור מתמשך.
- **5.3 תפקידים, אחריות וסמכויות ארגוניות** — הקצאה ותקשור ברורים של אחריות ל-ISMS (למשל CISO, בעלי נכסים/Asset Owners, בעלי סיכונים/Risk Owners).

סעיף 6 — תכנון (Planning)

- **6.1.1 טיפול בסיכונים והזדמנויות (כללי)** — תכנון פעולות להתמודדות עם סיכונים והזדמנויות שזוהו בסעיף 4.1-4.2.

- **6.1.2 הערכת סיכוני אבטחת מידע (Risk Assessment)** — קביעת מתודולוגיה עקבית וניתנת לחזרה להערכת סיכונים: זיהוי סיכונים, בעלי סיכונים, ניתוח (סבירות x השפעה), והערכה מול קריטריוני קבלת סיכון.
- **6.1.3 טיפול בסיכוני אבטחת מידע (Risk Treatment)** — בחירת אפשרויות טיפול (הפחתה, העברה, הימנעות, קבלה), בחירת בקורות מתוך נספח A (או מקורות אחרים), והפקת **הצהרת ההתאמה (Statement of Applicability – SoA)** — מסמך מפתח שמפרט אילו מ-93 הבקורות נבחרו, אילו הוחרגו ומדוע, ואיך כל בקרה מיושמת.
- **6.2 יעדי אבטחת מידע ותכנון להשגתם** — יעדים מדידים (SMART ככל האפשר), מתועדים, ומתוקשרים, עם תכנית ברורה: מה ייעשה, אילו משאבים, מי אחראי, מתי, ואיך יוערכו התוצאות.
- **6.3 תכנון שינויים** (נוסף במהדורת 2022) — כשה-ISMS זקוק לשינוי (מבני, היקפי, תהליכי), השינוי חייב להתבצע באופן מתוכנן ומבוקר.

סעיף 7 — תמיכה (Support)

- **7.1 משאבים** — הקצאת משאבים (אנשים, תקציב, תשתית, זמן) המספיקים להקמה, יישום, תחזוקה ושיפור מתמשך של ה-ISMS.
- **7.2 מיומנות (Competence)** — הבטחת שאנשים העובדים תחת שליטת הארגון מיומנים (השכלה, הכשרה, ניסיון) לתפקידים באבטחת מידע, כולל שמירת עדות מתועדת.
- **7.3 מודעות (Awareness)** — עובדים מודעים למדיניות אבטחת המידע, לתרומתם לאפקטיביות ה-ISMS, ולהשלכות אי-עמידה בדרישות.
- **7.4 תקשורת** — קביעת מתי, עם מי, איך, ומי מתקשר בנוגע לענייני ISMS — פנימית וחיצונית.
- **7.5 מידע מתועד (Documented Information)** — דרישות ליצירה, עדכון, ובקרה של מסמכי ה-ISMS: גרסאות, אישורים, זמינות, הגנה מפני אובדן/שימוש לא נאות.

סעיף 8 — תפעול (Operation)

- **8.1 תכנון ובקרה תפעולית** — יישום בפועל של התכניות מסעיף 6: תהליכים, קריטריונים לביצוע, בקרה על שינויים מתוכננים, סקירת השלכות של שינויים לא מתוכננים.
- **8.2 הערכת סיכוני אבטחת מידע** — ביצוע הערכות סיכונים בפרקי זמן מתוכננים או כאשר מתרחשים שינויים משמעותיים.
- **8.3 טיפול בסיכוני אבטחת מידע** — יישום תכנית הטיפול בסיכונים ושמירת עדות לתוצאות.

סעיף 9 — הערכת ביצועים (Performance Evaluation)

- **9.1 ניטור, מדידה, ניתוח והערכה** — קביעה מה יימדד, איך, מתי, ומי מנתח ומעריך את התוצאות (מדדי KPI לאבטחת מידע).
- **9.2 ביקורת פנימית (Internal Audit)** — תכנית ביקורות בפרקי זמן מתוכננים לבדיקה שה-ISMS תואם לדרישות התקן ולדרישות הארגון עצמו, ושהוא מיושם ומתוחזק באפקטיביות. הביקורת חייבת להיות אובייקטיבית ובלתי-משוחדת (מבקר לא מבקר את תחום עבודתו שלו).

• **9.3 סקר הנהלה (Management Review)** — ההנהלה הבכירה סוקרת את ה-ISMS בפרקי זמן מתוכננים (בד"כ שנתי לפחות), בוחנת קלט (תוצאות ביקורות, משוב מבעלי עניין, שינויי סיכונים, הזדמנויות לשיפור) ומפיקה החלטות ופעולות.

סעיף 10 — שיפור (Improvement)

• **10.1 שיפור מתמשך** — שיפור מתמיד של התאמת, נאותות ואפקטיביות ה-ISMS (נוסח שהתעדכן ב-2022 והפך תמציתי יותר).

• **10.2 אי-התאמה ופעולה מתקנת** — כאשר מתגלה אי-התאמה (Nonconformity), הארגון חייב להגיב אליה, לנקוט פעולה לתקן ולטפל בהשלכות, להעריך את הצורך בהסרת הסיבה השורשית (Root Cause) כדי שלא תישנה, ליישם כל פעולה נדרשת, ולסקור את אפקטיביות כל פעולה מתקנת.

הערה מתודולוגית: "חייב" מול "לפי הצורך"

ההבחנה בין סעיפים 4-10 (חובה מלאה, ללא אפשרות החרגה) לבין נספח A (רשימת בקורות שממנה בוחרים לפי סיכון) היא אחד הדברים הכי חשוב להבין נכון בתקן — טעות נפוצה היא לחשוב ש"ISO 27001" = "יישום כל 93 הבקורות". בפועל, יישום כל הבקורות בלי הצדקה מבוססת-סיכון הוא בעצמו סימן לחוסר הבנה של רוח התקן, שמבוסס על ניהול סיכונים ולא על צ'ק-ליסט טכני גורף.

מה מבקר בפועל מבקש לראות, סעיף-סעיף

טבלה זו מתרגמת כל סעיף לשאלה המעשית ולסוג העדות שמבקר (פנימי או חיצוני, ראו פרק 5) בדרך כלל יבקש כדי להשתכנע שהסעיף לא רק "כתוב" אלא גם "חי":

סעיף	השאלה המעשית	עדות טיפוסית
4.1-4.2	מי בעלי העניין שלכם ומה הם דורשים?	מסמך ניתוח הקשר/בעלי עניין, רשימת דרישות חוזיות/רגולטוריות
4.3	מה בדיוק בתוך ומה מחוץ ל-ISMS?	מסמך Scope חתום, דיאגרמת רשת/ארכיטקטורה תואמת
5.1	איך ההנהלה הבכירה מפגינה מעורבות בפועל?	פרוטוקולי ישיבות הנהלה, אישור תקציב/משאבים, נוכחות בסקר הנהלה
5.2	האם המדיניות מוכרת ולא רק קיימת?	מדיניות חתומה + עדות תקשורת/הדרכה עליה
6.1.2-6.1.3	איך הוערכו הסיכונים ולמה נבחרו הבקורות?	מתודולוגיית סיכונים מתועדת, רשומת סיכונים, SoA עם נימוקים
6.2	האם היעדים ניתנים למדידה בפועל?	יעדים מתועדים + נתוני מעקב בפועל (לא רק "היעד היה X")
7.2-7.3	האם אנשי מפתח מוכשרים ועובדים מודעים?	תיקי הכשרה, נוכחות בהדרכות מודעות, תוצאות תרגולי פישנינג

סעיף	השאלה המעשית	עדות טיפוסית
7.5	האם המסמכים מבוקרים (לא גרסאות ישנות מתרוצצות)?	מערכת בקרת גרסאות/היסטוריית שינויים למסמכי ISMS
8.1-8.3	האם התהליכים באמת פועלים כפי שתוכננו?	לוגים, טיקטים, ראיונות עם בעלי תפקידים
9.1	אילו מדדים נאספים ומה עושים איתם?	דוחות KPI תקופתיים, דיון עליהם בפגישות
9.2	האם הביקורת הפנימית אובייקטיבית ומבוצעת בפועל?	תכנית ביקורת שנתית, דוחות ביקורת פנימית עם ממצאים אמיתיים
9.3	האם ההנהלה באמת דנה בתוצאות?	פרוטוקול סקר הנהלה עם כל קלטי סעיף 9.3 והחלטות שנבעו ממנו
10.2	האם אי-התאמות נסגרות, לא רק מתועדות?	רשומת אי-התאמות עם ניתוח שורש, פעולה, ותאריך סגירה/אימות

דוגמה: קריטריוני סיכון ויעד מדיד

קריטריוני קבלת סיכון (6.1.2) — לדוגמה, סולם 1-5 לכל ציר (כפי שמיושם בכלי הנלווה לספר, פרק 6.3): ציון 1-4 מתקבל ללא אישור מיוחד; 5-9 דורש אישור בעל הנכס; 10-16 דורש אישור CISO ותכנית טיפול תוך רבעון; 17-25 דורש אישור הנהלה בכירה ותכנית טיפול מיידית (ימים עד שבועות).

דוגמה ליעד אבטחת מידע מדיד (6.2): "עד סוף הרבעון, 95% מהעובדים הקבועים ישלימו הדרכת מודעות אבטחת מידע שנתית, ואחוז ההקלקה על מיילי פשינג מדומים בתרגולים יירד מ-18% ל-10% לכל היותר" — יעד זה מדיד, בעל בעלים, בעל לוח זמנים, ומוסבר בתכנית פעולה (מי מריץ את התרגולים, באיזו תדירות, איך נמדד).

תיעוד מבוקר: מה זה אומר בפועל (7.5)

"מידע מתועד מבוקר" אינו דורש כלי ייעודי — גיליון Google Sheets, Confluence, או תיקיית Git עם היסטוריית קומיטים יכולים לספק את הבקרה הנדרשת, כל עוד מתקיימים בפועל: - **מזהה גרסה ותאריך עדכון אחרון** לכל מסמך מדיניות/נוהל. - **אישור (owner) מתועד** — מי אישר את הגרסה הנוכחית ומתי. - **שליטה בהפצה** — מי אמור לראות את המסמך (מדיניות ציבורית מול נוהל פנימי רגיש), ומניעת גישה למי שלא אמור. - **שמירת גרסאות היסטוריות** לצורך ביקורת ("מה המדיניות אמרה בזמן האירוע?").

פרק 3: נספח A – בקורות אבטחת מידע (מהדורת 2022)

נספח A של ISO/IEC 27001:2022 (התואם, מילה במילה, לגוף הבקורות של ISO/IEC 27002:2022) כולל **93 בקורות**, מאורגנות ב-4 קטגוריות (**themes**) במקום 14 הדומיינים של מהדורת 2013. חשוב לזכור: נספח A הוא **רשימה** לבחירה לפי סיכון – לא כל הבקורות רלוונטיות לכל ארגון, וההחלטה אילו לכלול/להחריג (ועם איזה נימוק) מתועדת ב-**הצהרת ההתאמה (Statement of Applicability, SoA)**.

בנוסף, כל בקרה במהדורת 2022 מתויגת ב-5 סוגי **תכונות (attributes)** שמאפשרים סינון וניתוח חוצה-קטגוריות: סוג הבקרה (מונעת/מגלה/מתקנת), מאפייני אבטחת מידע (סודיות/שלמות/זמינות), מושגי אבטחת סייבר (זיהוי/הגנה/גילוי/ תגובה/התאוששות לפי NIST CSF), יכולות תפעוליות (ממשל, ניהול נכסים, הגנה פיזית וכו'), ותחומי אבטחה (ממשל/אקוסיסטם, הגנה, הגנת מידע, המשכיות).

A.5 – בקורות ארגוניות (37 בקורות)

הקטגוריה הגדולה ביותר, עוסקת במדיניות, ממשל, ניהול סיכונים, נכסים, ספקים ותאימות. הרשימה המלאה (★ = בקרה חדשה שלא הייתה קיימת במהדורת 2013):

#	בקרה
5.1	מדיניות אבטחת מידע ומדיניות ספציפיות לנושא
5.2	תפקידים ואחריות באבטחת מידע
5.3	הפרדת תפקידים (Segregation of Duties)
5.4	אחריות ניהולית (Management responsibilities)
5.5	קשר עם רשויות
5.6	קשר עם קבוצות עניין ייעודיות (Special Interest Groups)
★ 5.7	מודיעין אימים (Threat Intelligence)
5.8	אבטחת מידע בניהול פרויקטים
5.9	מלאי נכסי מידע ונכסים נלווים
5.10	שימוש מקובל בנכסי מידע
5.11	החזרת נכסים (עם סיום העסקה/פרויקט)
5.12	סיווג מידע
5.13	תיג מידע (Labelling)
5.14	העברת מידע
5.15	בקרת גישה

#	בקרה
5.16	ניהול זהויות
5.17	מידע לאימות (Authentication Information)
5.18	זכויות גישה
5.19	אבטחת מידע ביחסי ספקים
5.20	אבטחת מידע בהסכמי ספקים
5.21	ניהול אבטחת מידע בשרשרת אספקת ה-ICT
5.22	ניטור, סקירה וניהול שינויים בשירותי ספקים
★ 5.23	אבטחת מידע בשימוש בשירותי ענן
5.24	תכנון והיערכות לניהול אירועי אבטחת מידע
5.25	הערכה והחלטה על אירועי אבטחת מידע
5.26	תגובה לאירועי אבטחת מידע
5.27	למידה מאירועי אבטחת מידע
5.28	איסוף ראיות
5.29	אבטחת מידע בזמן שיבוש (Business Continuity)
★ 5.30	מוכנות טכנולוגית מידע להמשכיות עסקית (ICT readiness)
5.31	דרישות משפטיות, סטטוטוריות, רגולטוריות וחוזיות
5.32	זכויות קניין רוחני
5.33	הגנת רשומות
5.34	פרטיות והגנה על מידע מזהה אישית (PII)
5.35	סקירה בלתי-תלויה של אבטחת מידע
5.36	עמידה במדיניות, בכללים ובתקנים לאבטחת מידע
5.37	נהלי הפעלה מתועדים

A.6 – בקורות אנשים (8 בקורות)

עוסקות במחזור חיי העובד ובגורם האנושי:

#	בקרה
6.1	בדיקות סינון (Screening) לפני העסקה
6.2	תנאי העסקה
6.3	מודעות, חינוך והכשרה לאבטחת מידע

#	בקרה
6.4	תהליך משמעותי
6.5	אחריות לאחר סיום/שינוי העסקה
6.6	הסכמי סודיות/אי-גילוי (NDA)
6.7	עבודה מרחוק (Remote Working)
6.8	דיווח על אירועי אבטחת מידע (על ידי עובדים)

A.7 – בקורות פיזיות (14 בקורות)

עוסקות באבטחה פיזית וסביבתית:

#	בקרה
7.1	היקפי אבטחה פיזית
7.2	כניסה פיזית
7.3	אבטחת משרדים, חדרים ומתקנים
★ 7.4	ניטור אבטחה פיזית
7.5	הגנה מפני איומים פיזיים וסביבתיים
7.6	עבודה באזורים מאובטחים
7.7	שולחן נקי ומסך נקי (Clear Desk / Clear Screen)
7.8	מיקום והגנה על ציוד
7.9	אבטחת נכסים מחוץ למתחם הארגון
7.10	מדיה אחסון (Storage Media)
7.11	תשתיות תומכות (חשמל, מיזוג וכו')
7.12	אבטחת כבילה
7.13	תחזוקת ציוד
7.14	סילוק או מיחזור בטוח של ציוד

A.8 – בקורות טכנולוגיות (34 בקורות)

הקטגוריה הטכנית – התחום המוכר ביותר לאנשי IT ואבטחת מידע:

#	בקרה
8.1	מכשירי קצה של משתמשים

#	בקרה
8.2	זכויות גישה מיוחסות (Privileged Access)
8.3	הגבלת גישה למידע
8.4	גישה לקוד מקור
8.5	אימות מאובטח (Secure Authentication)
8.6	ניהול קיבולת (Capacity Management)
8.7	הגנה מפני קוד זדוני
8.8	ניהול חולשות טכניות (Vulnerability Management)
★ 8.9	ניהול תצורה (Configuration Management)
★ 8.10	מחיקת מידע
★ 8.11	מיסוך נתונים (Data Masking)
★ 8.12	מניעת דליפת מידע (DLP)
8.13	גיבוי מידע
8.14	יתירות של מתקני עיבוד מידע
8.15	רישום אירועים (Logging)
★ 8.16	פעילויות ניטור (Monitoring Activities)
8.17	סנכרון שעונים
8.18	שימוש בתוכנות שירות (Utility Programs) פריבילגיות
8.19	התקנת תוכנה על מערכות תפעוליות
8.20	אבטחת רשתות
8.21	אבטחת שירותי רשת
8.22	הפרדת רשתות
★ 8.23	סינון אינטרנט (Web Filtering)
8.24	שימוש בקריפטוגרפיה
8.25	מחזור חיי פיתוח מאובטח (Secure Development Life Cycle)
8.26	דרישות אבטחת יישומים
8.27	עקרונות ארכיטקטורה והנדסה מאובטחת
8.28	קידוד מאובטח (Secure Coding)
8.29	בדיקות אבטחה בפיתוח ובקבלה
8.30	פיתוח במיקור-חוץ (Outsourced Development)
8.31	הפרדת סביבות פיתוח, בדיקה והפקה

#	בקרה
8.32	ניהול שינויים
8.33	מידע לבדיקות (Test Information)
8.34	הגנה על מערכות מידע בזמן ביקורת/בדיקה

סה"כ: $37 + 8 + 14 + 34 = 93$ בקרות, מתוכן **11 בקרות חדשות** (מסומנות ★) שלא היה להן מקבילה ישירה במהדורת 2013 (ראו השוואה מלאה בפרק 7).

העמקה בכמה מהבקורות החדשות (★) – מה זה אומר בפועל

- **5.7 מודיעין איומים:** לא חובה לרכוש פיד מודיעין מסחרי יקר – ברמת בסיס, עוקבים אחרי CVE/מודעות אבטחה רלוונטיות לטכנולוגיות שבשימוש (למשל mailing list של הפצת הלינוקס, יועצי CERT לאומיים), ומתעדים איך מידע זה משפיע על הערכת הסיכונים (6.1.2).
- **5.23 אבטחת מידע בשימוש בשירותי ענן:** דורש להבהיר, לכל שירות ענן, **היכן עובר קו הגבול באחריות המשותפת** (Shared Responsibility) מול הספק – ראו ISO/IEC 27017 בפרק 4.
- **8.9 ניהול תצורה:** תיעוד ותחזוקה של "תצורת בסיס" (baseline) מאובטחת למערכות/שרתים/מכולות, וזיהוי סטיות ממנה – ולא רק "הכל מוגדר איכשהו ועובד".
- **8.11 מיסוך נתונים:** שימוש בטכניקות כמו מיסוך, פסאודונימיזציה והכללה (generalization) כדי להגביל חשיפת מידע רגיש בסביבות לא-פרודקשן (בדיקות, פיתוח, אנליטיקס) – קשור ישירות ל-8.33 (מידע לבדיקות).
- **8.12 מניעת דליפת מידע (DLP):** לא בהכרח מוצר DLP ייעודי – יכול להתחיל מבקורות פשוטות (חסימת העלאת קבצים לאחסון ענן אישי, ניטור יציאות USB) לפני השקעה בפתרון ארגוני מלא.
- **8.16 פעילויות ניטור:** הבקרה המרכזית שקושרת את ה-ISMS ליכולת גילוי בפועל (Detection) – ולעיתים נבדקת ישירות בעזרת תרגילי Red/Purple Team (ראו פרק 9, סעיף 9.2).

תכונות הבקרה (Attributes) – דוגמה לשימוש

מעבר למיקום הבקרה בטבלה (5/6/7/8), כל בקרה מתויגת גם ב-5 צירי תכונה עצמאיים, שמאפשרים "לחתוך" את נספח A בדרכים אחרות – שימושי במיוחד לדוחות הנהלה ולתכנון תרגילי Red Team:

ציר תכונה	ערכים אפשריים
Control type	Preventive / Detective / Corrective
Information security properties	Confidentiality / Integrity / Availability
Cybersecurity concepts (NIST CSF)	Identify / Protect / Detect / Respond / Recover
Operational capabilities	..., Governance, Asset management, Physical security
Security domains	Governance & Ecosystem, Protection, Defence, Resilience

דוגמה לשימוש: אם רוצים להציג להנהלה "מה יש לנו בתחום הגילוי (Detect)?" — במקום לעבור ידנית על 93 בקורות, מסננים לפי $Cybersecurity\ concept = Detect$ ומקבלים בדיוק את הבקורות הרלוונטיות (בעיקר בקבוצת 8.15-8.16, לצד 5.25 ו-5.7). זהו אחד היתרונות המעשיים של מהדורת 2022 על פני 2013, שלא כללה תכונות כאלה כלל.

מיפוי בקורות בין 2013 ל-2022

93 הבקורות של 2022 אינן "מהמצאה מחדש" — רובן מיזוג ושילוב של בקורות ממהדורת 2013, לצד **11 בקורות חדשות לגמרי** (המסומנות למעלה): מודיעין איומים, ICT readiness להמשכיות עסקית, ניטור אבטחה פיזית, ניהול תצורה, מחיקת מידע, מיסוך נתונים, DLP, פעילויות ניטור, אבטחת ענן ביחסי ספקים, וסינון אינטרנט. טבלת מיפוי רשמית (Correlation Matrix) מפורסמת על ידי ISO ומסייעת לארגונים מוסמכים לעדכן את ה-SoA שלהם בעת מעבר בין מהדורות (ראו פרק 7).

עקרון מנחה: הבקרה היא כלי, לא מטרה

חשוב לזכור בכל בקרה: השאלה הנכונה אינה "האם יישמנו את הבקרה?" אלא "**האם הבקרה מטפלת בסיכון שזיהינו, באפקטיביות מספקת, ובאופן שניתן להוכיח בביקורת (עדות, לוגים, נהלים, תיעוד)?**". בקרה שמיושמת "כי כתוב בתקן" בלי קשר לסיכון בפועל היא לרוב סימן לגישה בירוקרטית ולא לניהול סיכונים אמיתי.

פרק 4: תקנים נגזרים ממשפחת ISO/IEC 27000

פרק זה סוקר את התקנים העיקריים במשפחת ISO/IEC 27000 מעבר ל-27001 עצמו — כולם תקני הנחיה, למעט 27701 שניתן להסמכה כתוספת (extension) ל-27001.

4.1 התקנים "הליבתיים" התומכים ישירות

ISO/IEC 27000 — סקירה ואוצר מילים

תקן מבוא **חינמי** (ISO מפרסמת אותו ללא תשלום) המספק סקירה כללית של משפחת 27000 ומילון מונחים מאוחד. נקודת פתיחה טובה למי שרוצה להבין את מפת הדרכים לפני שרוכשים תקנים ספציפיים.

ISO/IEC 27002 — קוד התנהגות לבקורות אבטחת מידע

"התאום" הישיר של 27001: מפרט **כיצד** ליישם כל אחת מ-93 הבקורות בנספח A — לכל בקרה יש מטרה (Purpose), הנחיה מפורטת ליישום, ולעיתים מידע נוסף. זהו המסמך שבו נעזרים בפועל בעת כתיבת נהלים ובקורות בשטח, בעוד ש-27001 קובע רק אילו בקורות לשקול ומה צריך להיות קיים ברמה הניהולית.

ISO/IEC 27003 — הנחיות ליישום

מדריך מעשי, שלב-אחר-שלב, להקמת ISMS: מ-buy-in של ההנהלה, דרך הגדרת היקף, הערכת סיכונים, ועד הכנה לביקורת הסמכה. שימושי במיוחד לארגונים שמיישמים לראשונה.

ISO/IEC 27004 — ניטור, מדידה, ניתוח והערכה

מרחיב את דרישת סעיף 9.1 של 27001: איך בונים תכנית מדידה, בוחרים מדדים (KPIs) משמעותיים לאבטחת מידע (לא רק "מספר אירועים" אלא מדדים שמראים אפקטיביות בקורות בפועל), ואיך מדווחים עליהם להנהלה.

ISO/IEC 27005 — ניהול סיכונים אבטחת מידע

התקן המרכזי התומך בסעיף 6.1.2 ו-8.2 (הערכת וטיפול בסיכונים). מספק מסגרת מושגית לתהליך ניהול סיכונים (זיהוי, ניתוח, הערכה, טיפול, ניטור, תקשורת), **מבלי לכפות מתודולוגיה ספציפית** — ארגון יכול לבחור בין גישות איכותניות, כמותיות, או היברידיות (לדוגמה, מודל Likelihood × Impact כפי שממומש בכלי [cwe_risk_calculator.py](#) הנלווה לספר זה). המהדורה העדכנית (2022) מיושרת מלאה עם ISO 31000 (ניהול סיכונים ארגוני כללי).

ISO/IEC 27006 – דרישות לגופי ביקורת והסמכה

תקן טכני המיועד **לגופי ההסמכה עצמם** (לא לארגונים המוסמכים) – קובע את הדרישות שגוף הסמכה (כמו BSI, DNV, TÜV, SGS ואחרים) חייב לעמוד בהן כדי להיות מוכר כמוסמך להעניק תעודות ISO/IEC 27001.

ISO/IEC 27007 – הנחיות לביקורת ISMS

מדריך למבקרים (פנימיים וחיצוניים) על ניהול תכנית ביקורת, תכנון ביקורת בודדת, ואיסוף עדות ביקורת – משלים את ISO 19011 (הנחיות ביקורת מערכות ניהול כלליות) עם היבטים ייחודיים ל-ISMS.

ISO/IEC TS 27008 – הנחיות למבקרי בקורות אבטחת מידע

ממוקד יותר מ-27007: איך בודקים **בפועל** שבקרה טכנית מיושמת כראוי (בדיקות טכניות, סקירת תצורה) – משלים בין ביקורת ניהולית (27007) לבדיקה טכנית.

4.2 תקני מגזר (Sector-specific)

ISO/IEC 27009 – יישום ספציפי למגזר

"מטא-תקן" שקובע **איך** מפתחים תקן ISMS ספציפי-מגזר (sector-specific) שמבוסס על 27001/27002 – למשל איך 27019 או 27799 (ראו למטה) הורשו להוסיף בקורות ייחודיות מבלי לסתור את הדרישות הבסיסיות.

ISO/IEC 27011 – טלקומוניקציה

התאמת בקורות 27002 לארגוני תקשורת (ITU-T X.1051), עם דגש על אבטחת תשתיות רשת קריטיות ושיתוף מידע בין מפעילים.

ISO/IEC 27019 – תעשיית האנרגיה

בקורות ייעודיות למערכות בקרה תעשייתיות (ICS/SCADA) בענף האנרגיה (חשמל, גז, נפט) – כולל היבטים ייחודיים כמו זמינות קריטית ומערכות legacy.

ISO/IEC 27799 – מידע בריאותי (Health Informatics)

מדריך ליישום 27002 בארגוני בריאות, מותאם לרגישות מידע רפואי ולדרישות כמו HIPAA (בארה"ב) ותקנות בריאות מקומיות.

4.3 תקני ענן ופרטיות

ISO/IEC 27017 – בקורות אבטחת מידע לשירותי ענן

מוסיף הנחיות ובקורות ספציפיות לענן על גבי 27002: **הבהרת אחריות** בין ספק הענן (Cloud Service Provider) ללקוח (Cloud Service Customer) לכל בקרה (מודל האחריות המשותפת – Shared Responsibility), ומוסיף 7 בקורות ייעודיות לענן (הפרדת סביבות וירטואליות, ניהול תצורת מכונות וירטואליות, פעולות ניהול ענן, ועוד).

ISO/IEC 27018 – הגנת מידע אישי (PII) בענן ציבורי

ממוקד בהגנת **מידע מזהה אישי (PII)** שמעובד על ידי ספקי ענן ציבורי בתפקיד מעבד (Processor) עבור לקוחותיהם. רלוונטי מאוד לתאימות ל-GDPR ולחוקי פרטיות דומים בעת רכישת שירותי ענן.

ISO/IEC 27701 – מערכת ניהול מידע פרטיות (PIMS)

התוספת (extension) המשמעותית ביותר ל-27001 – מרחיבה את ה-ISMS למערכת ניהול פרטיות מלאה (PIMS – Privacy Information Management System), עם דרישות נפרדות לארגונים בתפקיד **בקר מידע (Controller)** ולארגונים בתפקיד **מעבד מידע (Processor)**. ארגון שכבר מוסמך ל-27001 יכול "להרחיב" את ההסמכה שלו ל-27701 מבלי לבנות ISMS נפרד – משמש כלי מרכזי להפגנת תאימות ל-GDPR ולחוקי פרטיות אחרים (כולל חוק הגנת הפרטיות הישראלי המתקין).

4.4 תקני נושא (Topic-specific)

ISO/IEC 27013 – יישום משולב של 27001 ו-20000-1

מדריך מעשי לארגונים שכבר מיישמים ניהול שירותי IT (ITSM, מבוסס ITIL) לפי ISO/IEC 20000-1 ורוצים להוסיף ISMS מבלי לבנות שתי מערכות ניהול נפרדות ומקבילות – איך למפות תהליכים משותפים (ניהול שינויים, ניהול אירועים, ניהול ספקים) כך שישרתו את שני התקנים בו-זמנית.

ISO/IEC 27014 – ממשל אבטחת מידע (Governance)

עוסק ברמה שמעל ה-ISMS התפעולי: **אחריות הדירקטוריון וההנהלה הבכירה** לאבטחת מידע כנושא ממשל תאגידי, לא רק כנושא טכני-תפעולי. מגדיר עקרונות ממשל (הערכה, כיוון, ניטור – Evaluate/Direct/Monitor), בהשראת ISO/IEC 38500 לממשל IT) ומודל תקשורת בין ההנהלה הבכירה לפונקציית האבטחה. רלוונטי במיוחד לארגונים ציבוריים/נסחרים שבהם אחריות הדירקטוריון לסיכוני סייבר היא סוגיה רגולטורית עולה.

ISO/IEC 27016 — כלכלה ארגונית של אבטחת מידע

מסגרת לניתוח **עלות מול תועלת** של השקעות באבטחת מידע — עוזר לענות על "כמה זה שווה להשקיע בבקרה X?" בשפה שדירקטוריון/סמנכ"ל כספים מבינים (ROI, הפחתת סיכון כספי צפוי), ולא רק בשפה טכנית.

ISO/IEC 27021 — דרישות כשירות למקצועי ISMS

"מפת תפקידים" שמגדירה אילו ידע, מיומנויות וניסיון נדרשים מבעלי תפקידים שונים ב-ISMS (מוביל יישום, מבקר פנימי/מוביל ביקורת, יועץ) — שימושי כבסיס לתוכניות פיתוח מקצועי ולקביעת קריטריוני גיוס לתפקידי CISO/GRC.

ISO/IEC 27032 — הנחיות לאבטחת סייבר

משיק לאבטחת מידע אך מרחיב לזירת ה"סייברספייס" (Cyberspace) כמרחב דיגיטלי משותף שחוצה גבולות ארגוניים — עוסק בתיאום ושיתוף מידע בין ארגונים, ספקי שירות, וגורמי ממשל, מעבר לגבולות ה-ISMS של ארגון בודד. משיק ישירות לפרק 9 (Threat Intelligence, שיתוף IOCs).

ISO/IEC 27033 (סדרה, חלקים 1-6) — אבטחת רשתות

פירוט טכני-הנדסי לעיצוב, יישום ותפעול אבטחת רשתות: ארכיטקטורת רשת מאובטחת (חלק 1), תרחישי רשת ספציפיים כמו VPN וגישה מרחוק (חלקים 5-6). מרחיב משמעותית את 8.22-8.20 A.8.20 (אבטחת רשתות, הפרדת רשתות) לרמת פירוט הנדסי.

ISO/IEC 27034 (סדרה) — אבטחת יישומים (Application Security)

מסגרת מלאה ל-SDLC מאובטח ברמת ארגון (ONF — Organization Normative Framework) וברמת פרויקט בודד — מרחיב את 8.34-8.25 A.8.25. שימושי לארגוני פיתוח שרוצים סטנדרטיזציה של דרישות אבטחה בין צוותי פיתוח שונים, לא רק "כל צוות עושה משהו אחר".

ISO/IEC 27035 (חלקים 1-3) — ניהול אירועי אבטחת מידע

התקן המרכזי המרחיב את 5.28-5.24 A.5.24: עקרונות ניהול אירועים (חלק 1), הנחיות תפעוליות מפורטות לתגובה בפועל (חלק 2), והיבטי תיאום בין-ארגוני (חלק 3, כגון שיתוף מידע על אירועים בין חברות בענף). זהו התקן שכדאי להכיר לעומק אם בונים נוהל תגובה לאירועים אמיתי (IR Playbook), לא רק פסקה כללית ב-ISMS.

ISO/IEC 27036 (חלקים 1-4) — אבטחת מידע ביחסי ספקים

מרחיב את 5.22-5.19 A.5.19: עקרונות כלליים (חלק 1), דרישות ליחסי ספק-לקוח (חלק 2), אבטחת שרשרת אספקת ICT (חלק 3), ואבטחת שירותי ענן ספציפית (חלק 4, חופף חלקית ל-ISO/IEC 27017). קריטי לארגונים שתלויים בשרשרת אספקה מורכבת (ספקי תוכנה, קבלני משנה, מיקור-חוץ פיתוח).

משפחת הפורניזיקה הדיגיטלית – ISO/IEC 27037-27043, 27050

קבוצת תקנים שמפרטת כיצד לטפל בראיות דיגיטליות באופן שישרוד ביקורת משפטית: זיהוי, איסוף ושימור ראיות דיגיטליות (27037), עקרונות ניתוח ראיות (Investigation of incidents — 27042), הבטחת נאותות תהליך החקירה (Incident investigation principles and — 27043), וגילוי אלקטרוני/eDiscovery (27050), בעיקר רלוונטי בהליכים משפטיים/רגולטוריים). משיק ישירות ל-A.5.28 (איסוף ראיות) ולפרק 9 (שרשרת משמורת בממצאי pentest/תגובה לאירוע).

4.5 תקנים קרובים מחוץ למשפחת 27000

- **ISO 22301 – ניהול המשכיות עסקית (BCM):** תקן עצמאי (לא חלק פורמלי ממשפחת 27000, אך משיק ישירות ל-A.5.29-5.30). ארגונים רבים משלבים 27001 ו-22301 יחד.
- **ISO 31000 – ניהול סיכונים, עקרונות והנחיות:** המסגרת הארגונית הכללית לניהול סיכונים שעליה מבוסס ISO/IEC 27005.
- **ISO/IEC 20000-1 – ניהול שירותי IT:** תקן ITSM (מבוסס ITIL); ISO/IEC 27013 מסביר איך ליישם 27001 ו-20000-1 יחד ביעילות.
- **ISO 9001 – ניהול איכות:** התקן ה"אב" שממנו נגזר מבנה Annex SL המשותף.

4.6 טבלת סיכום מהירה

תקן	סוג	שימוש עיקרי
27001	דרישות (ניתן להסמכה)	ה-ISMS עצמו
27002	הנחיה	איך ליישם בקורות נספח A
27005	הנחיה	מתודולוגיית ניהול סיכונים
27017	הנחיה	בקורות ענן + אחריות משותפת
27018	הנחיה	הגנת PII בענן ציבורי
27701	דרישות (הרחבת הסמכה)	PIMS – ניהול פרטיות
27032	הנחיה	אבטחת סייבר
27035	הנחיה	ניהול אירועי אבטחת מידע
27036	הנחיה	אבטחת שרשרת ספקים
22301	דרישות (ניתן להסמכה, תקן נפרד)	המשכיות עסקית

4.7 מדריך החלטה מהיר: אילו תקנים באמת רלוונטיים לי?

עם +40 תקנים במשפחה, ארגון לא צריך "להכיר את כולם" – רק לענות על כמה שאלות פשוטות כדי לצמצם לרשימה הרלוונטית:

- **"אני מספק שירותי ענן, או קונה שירותי ענן קריטיים?"** → 27017 (ולקוחות ענן ששואלים על הגנת 27018 PII, ולעיתים 27036-4).

- **"אני מעבד/שולט במידע אישי (PII) ורוצה להוכיח תאימות פרטיות (GDPR וכו')?"** → 27701 היא ההרחבה המרכזית; שקלו הסמכה משולבת עם 27001 מההתחלה כדי לחסוך כפילות עבודה.

- **"יש לי SDLC פנימי ומוצרי תוכנה משלי?"** → 27034 ו-A.8.25-8.34 הם הליבה המעשית; פרק 9 (pentesting) הוא כלי האימות המרכזי לכך שה-SDLC אכן עובד.

- **"אני כבר מיישם ITSM (ITIL) לפי ISO/IEC 20000-1?"** → 27013 חוסך כפילות תיעוד ותהליכים.

- **"אני בענף מגזרי (אנרגיה/בריאות/טלקום)?"** → בדקו אם קיים תקן מגזר ייעודי (27019/27799/27011) לפני שממציאים בקורות מהתחלה.

- **"הדירקטוריון/רגולטור שואל על ממשל סייבר ברמת הנהלה?"** → 27014 נותן שפה ומסגרת מוכנה לדיווח לדירקטוריון.

- **"יש לי שרשרת ספקים/קבלני משנה מורכבת?"** → 27036 (כל 4 החלקים, לפי הרלוונטיות) משלים את A.5.19-5.22 ברמת פירוט תפעולי.

- **"אני צריך לבנות נוהל תגובה לאירועים אמיתי, לא רק פסקה ב-ISMS?"** → 27035 (שלושת החלקים) הוא ההשקעה המשתלמת ביותר מבין תקני הנושא.

הכלל המנחה: **אל תתחילו מרשימת התקנים – התחילו מהשאלה "מה מדאיג אותנו בפועל?" (סעיף 4.1-4.2 של 27001, פרק 2) ורק אז חפשו איזה תקן-הנחיה עוזר לענות על זה טוב יותר ממה שתמצאו לבד.**

פרק 5: תהליך ההסמכה והביקורת

5.1 מי מסמך את מי

ISO (הארגון הבינלאומי לתקינה) אינו מעניק תעודות הסמכה בעצמו. ההסמכה מתבצעת על ידי **גופי הסמכה (Certification Bodies)** עצמאיים — לדוגמה BSI, DNV, TÜV SÜD/Rheinland, SGS, Bureau Veritas ואחרים — שעצמם מוסמכים ("מוכרים") על ידי **גוף הכרה לאומי (Accreditation Body)** במדינתם (למשל UKAS בבריטניה, ANAB בארה"ב, או הרשות להסמכת מכונים בישראל). גוף ההסמכה פועל לפי דרישות **ISO/IEC 27006** (פרק 4).

בבחירת גוף הסמכה כדאי לוודא: - שהוא **מוכר (Accredited)** על ידי גוף הכרה מוכר בינלאומית (חבר ב-IAF — International Accreditation Forum), כדי שהתעודה תוכר בעולם. - שיש לו ניסיון בתחום/במגזר הארגון. - עלות ותנאי חוזה סבירים (הסמכה כרוכה במחויבות רב-שנתית, ראו למטה).

5.2 שלבי התהליך המלא

שלב 0: הכנה (לפני מעורבות גוף ההסמכה)

כולל את כל תהליך היישום המפורט בפרק 6 — הגדרת היקף, הערכת סיכונים, SoA, יישום בקורות, ביקורת פנימית, סקר הנהלה. **גוף ההסמכה לא מלווה את ההטמעה** — תפקידו לבדוק ולאשר (או להעריך יועץ חיצוני / מומחה פנימי מלווה את ההטמעה עצמה, בנפרד מגוף ההסמכה, כדי לשמור על עצמאות המבקר).

שלב 1: Stage 1 Audit (ביקורת שולחן / Documentation Review)

ביקורת ראשונית, בדרך כלל מרחוק או קצרה באתר, שבודקת: - שהתיעוד (מדיניות, נהלים, SoA, הערכת סיכונים) קיים ותואם בסיס לדרישות התקן. - מוכנות הארגון לביקורת Stage 2 (האם ה-ISMS "בשל" מספיק). - זיהוי פערים מהותיים מוקדם, לפני שמשקיעים בביקורת מלאה. בסיום Stage 1 מתקבל דו"ח עם ממצאים (אם יש) שיש לטפל בהם לפני Stage 2.

שלב 2: Stage 2 Audit (ביקורת יישום)

ביקורת מעמיקה באתר (או משולבת מרחוק, בהתאם למדיניות גוף ההסמכה), שבודקת: - **יישום בפועל** של הבקורות שנבחרו ב-SoA — לא רק תיעוד, אלא עדות אמיתית (לוגים, ראיונות עם עובדים, תצפית על תהליכים). - אפקטיביות ה-ISMS: האם התהליכים (ביקורת פנימית, סקר הנהלה, טיפול באי-התאמות) פעלו בפועל ולא רק "על הנייר". - ראיונות עם בעלי תפקידים מרכזיים (CISO, בעלי נכסים, IT, HR, לעיתים הנהלה בכירה).

בסיום, המבקר מסכם ממצאים שמסווגים ל: - **אי-התאמה חמורה (Major Nonconformity)** — כשל מהותי שמונע הסמכה עד לתיקון ואימות. - **אי-התאמה קלה (Minor Nonconformity)** —

פער נקודתי; לרוב לא חוסם הסמכה אך דורש תכנית תיקון ומעקב בביקורת הבאה. - **הזדמנות לשיפור (Observation)** — לא אי-התאמה, אך המלצה.

שלב 3: קבלת התעודה

לאחר סגירת אי-התאמות חמורות (וברוב המקרים גם קלות בתוך פרק זמן שנקבע), גוף ההסמכה מנפיק תעודה בתוקף ל-3 שנים.

שלב 4: ביקורות מעקב (Surveillance Audits)

בכל אחת מהשנתיים שלאחר ההסמכה (בד"כ שנה 1 ושנה 2) מתבצעת ביקורת מעקב חלקית — לא כל ה-ISMS נבדק מחדש, אלא מדגם של תחומים, כדי לוודא שהמערכת עדיין פעילה ומתוחזקת.

שלב 5: ביקורת חידוש (Recertification Audit)

בסוף השנה השלישית, ביקורת מלאה (בהיקף דומה ל-Stage 2) לחידוש התעודה לתקופה נוספת של 3 שנים.

5.3 ביקורת פנימית מול ביקורת הסמכה (חיצונית)

ביקורת פנימית (9.2)	ביקורת הסמכה (חיצונית)	
עובד/יועץ פנימי אובייקטיבי (לא מבקר את תחומו)	מבקר מוסמך מטעם גוף הסמכה מוכר	מי מבצע
לפי תכנית שקובע הארגון (בד"כ שנתיים)	Stage 1/2 בהסמכה ראשונית, מעקב שנתי, חידוש כל 3 שנים	תדירות
לספק להנהלה עדות עצמאית שה-ISMS תקין, לפני שהחיצוני מגיע	להעניק/לשמר תעודה רשמית מוכרת	מטרה
ממצאים פנימיים, קלט לסקר הנהלה	תעודת הסמכה / דו"ח אי-התאמות רשמי	תוצר

ביקורת פנימית טובה היא **קו ההגנה הראשון** לפני ביקורת חיצונית — ארגון שמזלזל בביקורת הפנימית ("רק כדי לסמן וי") לרוב מגלה זאת ביוקר בביקורת ההסמכה.

5.4 טעויות נפוצות בתהליך ההסמכה

- **הרצה לקראת הביקורת** — בניית תיעוד "יפה" רק סמוך למועד הביקורת, בלי שהתהליכים פעלו זמן מספיק כדי לצבור עדות אמיתית (ביקורת פנימית אחת, סקר הנהלה אחד — לא מספיקים "רגע לפני").
- **היקף (Scope) לא ריאלי** — היקף רחב מדי (כל הארגון, כל המערכות) כשאין משאבים לתחזק זאת, או היקף צר מדי שלא משקף את הסיכון האמיתי ומעורר שאלות אצל לקוחות/רגולטורים.
- **SoA שהוא "העתק-הדבק" מתבנית** — ללא קשר אמיתי להערכת הסיכונים של הארגון הספציפי.

- **חוסר מעורבות הנהלה בכירה** – סעיף 5.1 דורש מחויבות פעילה, לא רק חתימה על מדיניות.
- **בחירת גוף הסמכה לא מוכר (unaccredited)** – תעודה שלא תוכר על ידי לקוחות/שותפים בינלאומיים.

5.5 מי בפועל יושב מול הארגון בביקורת

- **מוביל ביקורת (Lead Auditor)** – אחראי כולל על תכנון הביקורת, ניהול הצוות (אם יש), וההחלטה הסופית על ההמלצה לתעודה.
- **מומחה טכני (Technical Expert)** – מצטרף לביקורת מורכבות (למשל תשתית ענן ייחודית, מערכות OT/ICS) כשנדרש ידע ספציפי שלמוביל הביקורת אין בהכרח.
- **מבקר-בפיתוח / עד (Witness Auditor)** – לעיתים גוף ההכרה (Accreditation Body) שולח משקיף כדי להעריך את **גוף ההסמכה עצמו** (לא את הארגון) – חלק משגרת הפיקוח על גופי הסמכה לפי ISO/IEC 27006.

5.6 כמה זמן וכסף זה בפועל דורש (הערכה כללית)

התקן/ISO לא קובעים תעריפון – משך הביקורת נגזר בעיקר ממספר העובדים בהיקף ה-ISMS, ממספר האתרים/מיקומים, וממידת המורכבות הטכנולוגית. **טווחי הערכה כלליים בלבד** (משתנים בין גופי הסמכה ובין מורכבות ספציפית):

גודל ארגון (בהיקף ה-ISMS)	סדר גודל ימי ביקורת (Stage 1+2 מצטבר)
עד כ-10 עובדים	2-3 ימים
כ-10-50 עובדים	3-5 ימים
כ-50-250 עובדים	5-8 ימים
מעל 250 עובדים / מרובה אתרים	תלוי היקף, לרוב +8 ימים ודיגמת אתרים (Sampling)

עלויות נגזרות מימי הביקורת (תעריף יומי של גוף ההסמכה), ומצטברות על פני מחזור ה-3 שנים: הסמכה ראשונית (Stage 1+2) + 2 ביקורות מעקב + ביקורת חידוש. **מעבר לעלות גוף ההסמכה עצמו**, יש לתקצב גם: זמן עבודה פנימי (השעות היקרות ביותר בפועל), ולעיתים יועץ חיצוני להטמעה, וכלי GRC (סעיף 6.3 בפרק 6).

5.7 ביקורת רב-אתרית ו-Sampling

ארגון עם כמה סניפים/מיקומים בהיקף ה-ISMS לא בהכרח מקבל ביקורת מלאה בכל אתר בכל פעם – גופי הסמכה מיישמים **דיגמת אתרים (Site Sampling)** לפי כללי IAF: מדגם מייצג של אתרים נבדק בכל מחזור, כשקריטריונים לבחירת המדגם כוללים דמיון בין האתרים (אותם תהליכים/בקורות) והיסטוריית ממצאים. אתר עם היסטוריית אי-התאמות חמורות בדרך כלל לא "מדולג" במדגם.

5.8 ערעור ותלונה

לארגון יש זכות לערער על החלטת גוף ההסמכה (למשל אי-הענקת תעודה, או דירוג אי-התאמה שהארגון חולק עליו) – תהליך הערעור (Appeal) מוגדר בנהלי גוף ההסמכה עצמו (חלק מדרישות ISO/IEC 27006). באופן דומה, קיים מנגנון תלונה (Complaint) גם כלפי התנהלות המבקר עצמו. שווה לברר את תהליכי הערעור/תלונה **לפני** חתימה על חוזה עם גוף הסמכה, לא אחרי בעיה.

פרק 6: יישום מעשי בארגון

פרק זה מציג מפת דרכים מעשית להטמעת ISMS לפי ISO/IEC 27001, מהחלטה ראשונית ועד ביקורת הסמכה, עם דגש על הכלים הפרקטיים שהופכים את התקן ממסמך תיאורטי לתהליך עבודה.

6.1 מפת דרכים להטמעה (סדר מומלץ)

1. **גיוס תמיכת הנהלה בכירה (Sponsorship)** — ללא זה, סעיף 5.1 ייכשל מלכתחילה. יש להציג עלות מול תועלת (עסקית: דרישת לקוחות/מכרזים, הפחתת סיכון, יתרון תחרותי; לא רק "כי צריך").
2. **הגדרת היקף (Scope)** — בחירת מוצרים/שירותים/יחידות/מיקומים שייכללו. כלל אצבע: התחילו בהיקף ממוקד (למשל מוצר SaaS אחד, לא כל החברה), הרחיבו בהמשך.
3. **מינוי אחראי (ISMS (CISO/Information Security Manager)** ותפקידים נלווים: בעלי נכסים, בעלי סיכונים.
4. **מיפוי נכסי מידע** — מערכות, מאגרי מידע, תשתיות, ספקים קריטיים — הבסיס להערכת הסיכונים.
5. **הערכת סיכונים (Risk Assessment)** — לפי מתודולוגיה עקבית (ISO/IEC 27005, פרק 4). כאן נכנס לשימוש כלי כמו `cwe_risk_calculator.py` הנלווה לספר, כדוגמה למודל ניקוד $Likelihood \times Impact$ עבור חולשות טכניות מזוהות (למשל מתוצאות סקירת קוד, בדיקות חדירה, או סריקות פגיעויות שממופות ל-CWE) — ראו 6.3 למטה.
6. **טיפול בסיכונים ובניית ה-SoA** — בחירת בקורות מנספח A (או מקורות נוספים), תיעוד הנמקה לכל בקרה שנכללה/הוחרגה.
7. **יישום הבקורות** — הפרויקט "הכבד" בפועל: מדיניות, נהלים, שינוי תהליך, הטמעת בקורות טכניות.
8. **הכשרה ומודעות (7.2-7.3)** — לכל העובדים הרלוונטיים, לא רק ל-IT.
9. **הפעלה שוטפת ואיסוף עדות** — לתת ל-ISMS "לרוץ" מספיק זמן (מומלץ לפחות 2-3 חודשים) כדי לצבור לוגים, רישומי אירועים, בקשות גישה וכו', לפני ביקורת פנימית.
10. **ביקורת פנימית (9.2)** — בדיקה עצמאית שהכול פועל כמתוכנן.
11. **סקר הנהלה (9.3)** — ההנהלה סוקרת תוצאות, מקבלת החלטות.
12. **טיפול באי-התאמות שעלו (10.2)** לפני הביקורת החיצונית.
13. **ביקורת הסמכה (פרק 5)**.

6.2 מבנה תיעוד מומלץ

רמה	מסמכים לדוגמה
1. מדיניות (Policy)	מדיניות אבטחת מידע ראשית, מדיניות פרטיות
2. נהלים (Procedures)	ניהול סיכונים, ניהול אירועים, ניהול גישה, ניהול שינויים, המשכיות עסקית
	הקשחת שרתים, נוהל גיבוי טכני, checklist onboarding/offboarding

רמה	מסמכים לדוגמה
3. הנחיות עבודה (Work Instructions)	
4. רשומות (Records)	דוחות ביקורת פנימית, פרוטוקולי סקר הנהלה, רשומת סיכונים, יומני אירועים

מסמך-העל המקשר בין הכול הוא ה-**SoA**: לכל בקרה — האם נכללה, מקור ההחלטה (תוצאת הערכת סיכונים / דרישה חוזית / דרישה רגולטורית), וקישור לנוהל/בקרה המיישמים אותה.

6.3 שימוש בכלי מחשבון הסיכונים המבוסס-CWE

בעת הערכת סיכונים לרכיבי תוכנה (יישומי Web, שירותי API, קוד פנימי), נפוץ שמקור הממצאים הוא כלי סריקה סטטית/דינמית, מבדק חדירה, או ביקורת קוד — וממצאים אלו לרוב ממופים למזהי **CWE** (Common Weakness Enumeration) סטנדרטיים. הכלי הנלווה `iso27001-book/tools/cwe_risk_calculator.py` ממחיש כיצד להפוך רשימת ממצאי CWE גולמית לרשומת סיכונים מדורגת, בהתאם למתודולוגיית ISO/IEC 27005 שתוארה בפרק 4:

```
# ים נתמכים בכלי, כולל ערכי ברירת מחדל-CWE הצגת רשימת
python3 iso27001-book/tools/cwe_risk_calculator.py list

# חישוב רשומת סיכונים מלאה מקובץ ממצאים, ממוינת מהחמור לקל
python3 iso27001-book/tools/cwe_risk_calculator.py report \
--input iso27001-book/tools/sample_findings.json \
--output risk_register.csv
```

התוצאה (רשומת סיכונים ממוינת עם רמת סיכון Low/Medium/High/Critical) יכולה לשמש כקלט ישיר לתכנית טיפול בסיכונים (6.1.3) — למשל: סיכונים ברמת Critical מטופלים לפני ההסמכה, סיכוני Medium מקבלים תכנית טיפול עם יעד זמן, וסיכוני Low מתועדים כקבלת סיכון (Risk Acceptance) עם אישור בעל הסיכון. **חשוב לשוב ולהדגיש** (כמפורט ב- `tools/README.md`): ערכי ברירת המחדל בכלי הם דוגמה פדגוגית ולא הערכה מדויקת של סיכון בפועל — יש לכייל likelihood / impact לפי ההקשר האמיתי (חשיפה לאינטרנט, רגישות הנתונים, בקורות מפצות קיימות).

6.4 מדדים (KPIs) לדוגמה

- זמן ממוצע לזיהוי אירוע אבטחה (MTTD) וזמן ממוצע לתגובה (MTTR).
- אחוז עובדים שהשלימו הדרכת מודעות שנתית.
- אחוז חולשות קריטיות/גבוהות שנסגרו בתוך SLA שהוגדר (למשל 30/90 יום).
- מספר אי-התאמות פתוחות מביקורת פנימית/חיצונית, לפי חומרה.
- אחוז נכסים עם בעלים (Asset Owner) מוגדר במלאי הנכסים.

6.5 תרבות אבטחת מידע – לא רק תיעוד

התקן דורש תיעוד, אך תיעוד לבדו לא מייצר אבטחה. ISMS אפקטיבי דורש: - **מעורבות הנהלה אמיתית**, לא רק חתימה פורמלית. - **תקשורת שוטפת** (לא רק הדרכה שנתית חד-פעמית) – עדכוני איומים, תרגולי פשיג, שיתוף לקחים מאירועים. - **תהליך דיווח אירועים ידיוותי** – עובדים שמפחדים לדווח על טעות (למשל לחצו על קישור חשוד) מעכבים גילוי מוקדם. - **שילוב אבטחה ב-SDLC** ולא כשכבה שמתווספת בסוף (Shift Left).

6.6 שילוב עם מערכות ניהול אחרות

ארגון שכבר מיישם ISO 9001 (איכות) או ISO 22301 (המשכיות עסקית) יכול לבנות **Integrated Management System (IMS)** בזכות מבנה Annex SL המשותף – סעיפי 4-10 כמעט זהים במבנה, מה שמפחית כפילות בתיעוד (למשל: מדיניות אחת מאוחדת, תהליך ביקורת פנימית אחד שמכסה כמה תקנים, סקר הנהלה משולב).

6.7 תפקידים ואחריות – טבלת RACI לדוגמה

R = Responsible (מבצע), A = Accountable (אחראי סופי), C = Consulted (מתייעצים איתו), I = Informed (מעודכן). דוגמה למיפוי בפרויקט הטמעה טיפוסי (יש להתאים לגודל/מבנה הארגון בפועל):

פעילות	הנהלה בכירה	CISO / מוביל ISMS	בעל נכס/מערכת	/IT DevOps	HR
אישור מדיניות אבטחת מידע (5.2)	A	R	I	I	I
הגדרת היקף ה- (4.3) ISMS	A	R	C	C	I
הערכת סיכונים (6.1.2)	I	A	R	C	I
בניית SoA (6.1.3)	I	A/R	C	C	I
יישום בקורות טכניות (נספח A.8)	I	C	A	R	I
הכשרת מודעות (7.3)	I	A	I	I	R
ביקורת פנימית (9.2)	I	C	I	I	I
סקר הנהלה (9.3)	A/R	R	C	I	I

(שורת "ביקורת פנימית" מכוונת מכוון ל-R חיצוני ל-CISO/מוביל ה-ISMS – עקרון האובייקטיביות מ-9.2 אומר שמי שאחראי לתחום לא יכול לבקר את עצמו.)

6.8 ציר זמן לדוגמה (ארגון בגודל בינוני, ~50-150 עובדים)

שלב	משך משוער	חופף לסעיף בפרק 2
Sponsorship + הגדרת היקף	שבועיים-חודש	5.1, 4.3
מיפוי נכסים + הערכת סיכונים ראשונית	3-5 שבועות	6.1.2
בניית SoA + תכנית טיפול בסיכונים	2-3 שבועות	6.1.3
יישום בקורות (המשימה הכבדה ביותר)	2-4 חודשים	נספח A
הכשרה ומודעות לכלל העובדים	מקביל לשלב הקודם	7.3-7.2
הרצה שוטפת + איסוף עדות	2-3 חודשים	8.3-8.1
ביקורת פנימית + סקר הנהלה	2-4 שבועות	9.3-9.2
טיפול באי-התאמות שעלו	2-4 שבועות	10.2
סה"כ עד מוכנות ל-Stage 1	כ-6-9 חודשים	

לוחות זמנים אלו הם סדר גודל בלבד — ארגון עם ISMS "בשל" יחסית (תהליכי IT מסודרים, תיעוד קיים) יכול לקצר משמעותית; ארגון שמתחיל מאפס (בלי נהלים, בלי מלאי נכסים) עלול לקחת יותר זמן, במיוחד בשלב יישום הבקורות.

6.9 כלים לתמיכה בתהליך (קטגוריות, לא המלצה על מוצר ספציפי)

- **גיליונות/Wiki פשוטים** (Google Sheets, Confluence, Notion) — מספיקים לחלוטין לארגונים קטנים-בינוניים; העלות הנמוכה ביותר, אך דורשים משמעת עצמית בשמירת גרסאות ובעלות (7.5).
- **פלטפורמות GRC ייעודיות** (קטגוריה כללית שכוללת כלים כמו Vanta, Drata, OneTrust, ZenGRC ואחרים) — מאטמות חלק מאיסוף העדות (חיבור ל-AWS/GCP/Okta לניטור בקורות רציף), מתאימות לארגונים גדולים יותר או לכאלה שרוצים "הסמכה מהירה" עם עדות רציפה במקום איסוף ידני.
- **הכלי הנלווה לספר** (`cwe_risk_calculator.py`, סעיף 6.3) — דוגמה לגישה ההפוכה: כלי CLI פשוט, קוד-פתוח, ללא תלות בספק חיצוני, שממחיש עיקרון (ניקוד סיכונים) שאפשר להטמיע גם בכלים גדולים יותר.

הבחירה בין הקטגוריות היא בעיקר פונקציה של גודל ארגון, תקציב, ומספר המערכות שצריך לנטר ברציפות — לא של "מה שמקובל", וגיליון אקסל מתוחזק היטב עדיף על כלי GRC יקר שאף אחד לא מעדכן.

פרק 7: 2013 מול 2022 – מה השתנה

7.1 שינויים בגוף התקן (סעיפים 4-10)

השינויים בדרישות עצמן (בניגוד לנספח A) מתונים יחסית – בעיקר ניסוח מחדד וסעיף חדש אחד:

- **סעיף 4.2** – הרחבה לכלול "אילו מהדרישות [של בעלי עניין] ייענו דרך ה-ISMS" (הבהרה שלא כל דרישת בעל עניין חייבת להיכנס להיקף ה-ISMS).
- **סעיף 4.4** – תוספת ניסוחית: "כולל את התהליכים הנדרשים ואת האינטראקציות ביניהם", דגש מוגבר על ראייה תהליכית.
- **סעיף 6.1.3** – הבהרת ניסוח בנוגע להשוואת בקורות שנבחרו מול נספח A.
- **סעיף 6.2** – הוספת דרישה שיעדי אבטחת מידע יהיו **ניתנים לניטור** (monitored) – לא רק מתועדים.
- **סעיף 6.3** – "תכנון שינויים" (Planning of changes) – **סעיף חדש לגמרי**. דורש שכאשר ה-ISMS זקוק לשינוי (למשל שינוי היקף, אימוץ טכנולוגיה חדשה, שינוי ארגוני), השינוי יתבצע בצורה מתוכננת ומבוקרת, ולא אד-הוק.
- **סעיף 7.4** – פישוט קל בניסוח דרישות התקשורת.
- **סעיף 9.1, 9.2, 9.3** – מבנה מחודד יותר (חלוקה ל-9.2.1/9.2.2 ו-9.3.1/9.3.2/9.3.3), בעיקר להבהרה ולא לתוספת דרישה מהותית.
- **סעיף 10** – **הוחלף הסדר**: ב-2013 "אי-התאמה ופעולה מתקנת" (10.1) קדם ל"שיפור מתמשך" (10.2). ב-2022 הסדר ההפך (10.1 שיפור מתמשך, 10.2 אי-התאמה), וניסוח שיפור מתמשך התקצר משמעותית.

המסקנה המעשית: ארגון שכבר מוסמך ל-2013 לא צריך לבנות ISMS מחדש – עיקר העבודה במעבר היא **עדכון ה-SoA וההערכה מחדש של הבקורות** (ראו 7.2), ותוספת תהליך פורמלי לניהול שינויים ב-(6.3) ISMS.

7.2 השינוי המרכזי: ריענון נספח A

	2013	2022
מספר בקורות	114	93
מספר קטגוריות	14 (A.5-A.18)	4 (ארגוניות, אנשים, פיזיות, טכנולוגיות)
בקורות חדשות	—	11 בקורות חדשות (ראו פרק 3)
בקורות שמוזגו	—	כ-57 בקורות מ-2013 מוזגו לתוך פחות בקורות ב-2022
בקורות ששונה שמן בלבד	—	23 בקורות
בקורות ללא שינוי	—	35 בקורות
תכונות (Attributes)	לא היו	נוספו 5 סוגי תכונות לכל בקרה (סעיף פרק 3)

הירידה ממספר גדול של קטגוריות צרות (14) לארבע קטגוריות רחבות משקפת מגמה כללית ב-ISO להפוך את הנספח לניתן-לניווט יותר, ולהדגיש קשרים חוצי-נושא (objectives) דרך התכונות ולא רק דרך מיקום הבקרה במסמך.

7.3 מה זה אומר לארגון מוסמך קיים

ISO קבעה **תקופת מעבר** (transition period) – ארגונים מוסמכים ל-ISO/IEC 27001:2013 נדרשו לעבור ל-2022 עד למועד יעד שקבע ה-IAF (בפועל, רוב גופי ההסמכה דרשו מעבר עד סוף 2025, במסגרת ביקורת מעקב או חידוש רגילה – ולא דרשו ביקורת מיוחדת נפרדת). מי שעדיין מחזיק בהסמכת 2013 בתוקף אחרי מועד היעד – ההסמכה כבר אינה תקפה להכרה בינלאומית.

שלבי המעבר המומלצים לארגון מוסמך: 1. מיפוי (Correlation Matrix) בין 114 הבקורות הישנות ל-93 החדשות (מפורסם על ידי ISO ועל ידי גופי ההסמכה). 2. עדכון ה-SoA לפי המבנה החדש (4 קטגוריות, כולל תכונות אם רוצים לנצל אותן לניתוח). 3. **הערכת סיכונים ממוקדת** ל-11 הבקורות החדשות (במיוחד: מודיעין איומים, ICT readiness להמשכיות עסקית, ניהול תצורה, מחיקת מידע, מיסוך נתונים, DLP, אבטחת ענן ביחסי ספקים) – האם ואיך הן רלוונטיות לארגון. 4. עדכון מדיניות ונהלים בהתאם לבקורות החדשות שנבחרו. 5. שילוב הביקורת בתהליך ביקורת המעקב או החידוש הקרובה (אין צורך בביקורת מיוחדת נפרדת, בכפוף למדיניות גוף ההסמכה הספציפי).

7.4 עדכון מקביל: ISO/IEC 27002

ISO/IEC 27002:2022 פורסם **לפני** 27001:2022 (בפברואר 2022, מול אוקטובר 2022), משום שהוא הבסיס שממנו "יוצא" נספח A. כל ארגון שמעדכן SoA צריך להיעזר ב-27002:2022 (לא 2013) לצורך ההנחיה המפורטת ליישום כל בקרה.

7.5 דוגמה קונקרטית למיפוי (Correlation) – תחום בקרת הגישה

כדי להמחיש איך "מיזוג בקורות" נראה בפועל, הנה דוגמה מתחום בקרת הגישה, שבמהדורת 2013 היה פזור על פני דומיין שלם (Access Control – A.9) ובמהדורת 2022 פוצל בין קטגוריית הארגון לקטגוריית הטכנולוגיה:

בקורות 2013 (דומיין A.9)	→	בקורות 2022 המקבילות
A.9.1.1 Access control policy	→	5.15 בקרת גישה
A.9.1.2 Access to networks and network services	→	8.20 אבטחת רשתות
A.9.2.1-A.9.2.6 (רישום משתמשים, ניהול זכויות, סקירת זכויות, הסרת/התאמת זכויות)	→	5.16 ניהול זכויות, 5.18 זכויות גישה
A.9.2.3 Management of privileged access rights	→	8.2 זכויות גישה מיוחדות
A.9.3.1 Use of secret authentication information	→	5.17 מידע לאימות
A.9.4.1 Information access restriction	→	8.3 הגבלת גישה למידע

בקורות 2013 (דומיין A.9)	→	בקורות 2022 המקבילות
A.9.4.2 Secure log-on procedures	→	8.5 אימות מאובטח
A.9.4.5 Access control to program source code	→	8.4 גישה לקוד מקור

שימו לב לתבנית: 8 תת-בקורות מדומיין A.9 הצטמצמו ל-8 בקורות חדשות, אך מפוזרות עכשיו בין קטגוריה ארגונית (x.5 – "מה המדיניות/מי מחליט") לקטגוריה טכנולוגית (x.8 – "איך זה מיושם בפועל"). זהו הדפוס החוזר ברוב תהליך המיזוג: הפרדה ברורה יותר בין **מדיניות/החלטה ליישום טכני**, ולא בהכרח איבוד תוכן.

7.6 טעות נפוצה במעבר: "העתק-הדבק" של SoA ישן

טעות שכיחה בארגונים שעוברים מ-2013 ל-2022 היא להתייחס למעבר כאל **פעולת מיפוי טכנית גרידא** – לקחת את ה-SoA הישן, להריץ אותו דרך טבלת ה-Correlation, ולהחליף מספרים. זה מפספס בדיוק את הסיבה שה-SoA קיים מלכתחילה: ההזדמנות לשאול מחדש, במיוחד לגבי 11 הבקורות החדשות, "האם זה רלוונטי **לנו**?" – ולא רק "מה המספר החדש של הבקרה הישנה?". ארגון שמדלג על שלב 3 שתואר למעלה (6.1.2 בפרק 2 – הערכת סיכונים ממוקדת לבקורות החדשות) עלול להגיע לביקורת המעבר עם SoA "מעודכן טכנית" אך לא משקף בפועל את סיכוני הענן/התצורה/הדליפה הרלוונטיים לארגון הספציפי.

פרק 8: מילון מונחים ורשימת בדיקה

8.1 מילון מונחים

מונח	הסבר
ISMS	Information Security Management System — מערכת ניהול אבטחת מידע
Asset (נכס)	כל דבר בעל ערך לארגון: מידע, מערכת, שירות, אנשים, מוניטין
Asset Owner (בעל נכס)	האחראי הארגוני על נכס מסוים, כולל אחריות להערכת הסיכון שלו
Threat (איום)	גורם פוטנציאלי לאירוע לא רצוי שעלול לפגוע בארגון
Vulnerability (חולשה)	חולשה בנכס שאיום יכול לנצל (לעיתים ממופה ל-CWE ברמה הטכנית)
Risk (סיכון)	השפעה של אי-ודאות על יעדים; לרוב מחושב כפונקציה של סבירות x השפעה
Likelihood (סבירות)	ההסתברות שאיום ינצל חולשה בפועל
Impact (השפעה)	היקף הנזק הפוטנציאלי אם הסיכון יתממש
Risk Treatment (טיפול בסיכון)	הפחתה (Mitigate), העברה (Transfer), הימנעות (Avoid), קבלה (Accept)
Risk Owner (בעל סיכון)	האחראי לקבל החלטה על אופן הטיפול בסיכון מסוים ולאשר קבלתו
SoA — Statement of Applicability	הצהרת ההתאמה: מסמך המפרט אילו בקורות נבחרו/הוחרגו ומדוע
Residual Risk (סיכון שייר)	הסיכון שנותר לאחר יישום בקורות טיפול
Control (בקרה)	אמצעי (טכני, ניהולי, פיזי) לשינוי סיכון
Nonconformity (אי-התאמה)	אי-עמידה בדרישה של התקן או של הארגון עצמו
Corrective Action (פעולה מתקנת)	פעולה לסילוק סיבת אי-התאמה כדי למנוע הישנות
Management Review (סקר הנהלה)	סקירה תקופתית של ה-ISMS על ידי ההנהלה הבכירה
Internal Audit (ביקורת פנימית)	ביקורת עצמאית בתוך הארגון לבדיקת תאימות ואפקטיביות
Certification Body (גוף הסמכה)	גוף חיצוני מוכר שמעניק תעודת הסמכה לתקן
Accreditation Body (גוף הכרה)	גוף לאומי שמכיר בגופי הסמכה
Stage 1 / Stage 2 Audit	שני שלבי ביקורת ההסמכה הראשונית (תיעוד, ואז יישום)

מונח	הסבר
Surveillance Audit (ביקורת מעקב)	ביקורת חלקית שנתיית בין ביקורות הסמכה/חידוש
PIMS	Privacy Information Management System — הרחבת ISMS לפי ISO / IEC 27701
PII	Personally Identifiable Information — מידע מזהה אישית
CIA Triad	Confidentiality, Integrity, Availability — משולש אבטחת המידע
CWE	Common Weakness Enumeration — קטלוג סטנדרטי של סוגי חולשות תוכנה
Annex SL	המבנה האחיד המשותף לכל תקני מערכות הניהול של ISO
PDCA	Plan-Do-Check-Act — מודל השיפור המתמשך של דמינג
Pentest (בדיקת חדירה)	ניסיון ידני מורשה לנצל חולשות בפועל, בשונה מסריקה אוטומטית (ראו פרק 9)
ROE — Rules of Engagement	מסמך המגדיר בכתב מה מותר/אסור לבדוק, טווח, חלון זמן, ואיש קשר לעצירת חירום
Black / Grey / White Box	רמת המידע המוקדם שמקבל בודק החדירה על המערכת הנבדקת
Red Team	סימולציית תוקף ריאליסטית וארוכת-טווח, לרוב ללא ידיעת צוות ההגנה (SOC)
PTES / OWASP Testing Guide / OSSTMM	מתודולוגיות מוכרות לביצוע בדיקות חדירה (ראו פרק 9.3)
Retest (בדיקה חוזרת)	אימות לאחר תיקון שממצא pentest אכן נסגר בפועל
Conformity (התאמה)	עמידה בדרישה (ההפך מ-Nonconformity)
Objective Evidence (עדות אובייקטיבית)	נתונים התומכים בקיום/אמינות דבר-מה (לוג, מסמך חתום, תצפית) — הבסיס לכל ממצא ביקורת
Interested Party (בעל עניין)	אדם/ארגון שיכול להשפיע, להיות מושפע, או לתפוס עצמו כמושפע מהחלטה/פעילות של הארגון (סעיף 4.2)
Top Management (הנהלה בכירה)	האדם/קבוצת האנשים המכוונים ומבקרים את הארגון ברמה הגבוהה ביותר
Effectiveness (אפקטיביות)	המידה שבה פעילויות מתוכננות מבוצעות ותוצאות מתוכננות מושגות בפועל — לא רק "בוצע" אלא "עבד"
Continual Improvement (שיפור מתמשך)	פעילות חוזרת לשיפור ביצועי ה-ISO (10.1)
Risk Criteria (קריטריוני סיכון)	התנאים שלפיהם מוערכת חשיבות סיכון — כולל קריטריון קבלת סיכון
Risk Appetite (תיאבון סיכון)	היקף וסוג הסיכון שהארגון מוכן לשאת כדי להשיג את יעדיו
	ההצהרה המתארת מה בקרה אמורה להשיג (לרוב ברמת נספח A/27002)

מונח	הסבר
Control Objective (יעד בקרה)	
Information Security Event (אירוע)	התרחשויות המעידה על הפרה אפשרית של מדיניות/בקרה, או מצב לא-ידוע רלוונטי – לא בהכרח פגע בפועל
Information Security Incident (תקרית)	אירוע (או סדרת אירועים) שיש לו סבירות משמעותית לפגוע בפעילויות העסקיות ולאיים על אבטחת המידע – תקרית = אירוע שהוסלם , לא כל אירוע הוא תקרית
CVSS	Common Vulnerability Scoring System – שיטת ניקוד חומרה סטנדרטית לחולשות ספציפיות (ראו פרק 9.6)
OWASP Top 10	רשימת 10 קטגוריות הסיכון הנפוצות ביותר ביישומי Web, מתעדכנת מעת לעת (ראו פרק 9.6)

8.2 רשימת בדיקה (Checklist) ליישום והסמכה

שלב הכנה

- [] קיבלנו מחויבות מפורשת ומתועדת מההנהלה הבכירה
- [] מונה אחראי ISMS (CISO / Information Security Manager)
- [] הוגדר היקף ה-ISMS בכתב (מוצרים, שירותים, מיקומים, נכסים)
- [] זהו כל בעלי העניין הרלוונטיים ודרישותיהם (חוזיות, רגולטוריות)

מדיניות ותכנון

- [] נכתבה ואושרה מדיניות אבטחת מידע
- [] הוגדרו תפקידים ואחריות (כולל בעלי נכסים ובעלי סיכונים)
- [] נבנה מלאי נכסי מידע מלא ומעודכן
- [] הוגדרה ותועדה מתודולוגיית הערכת סיכונים
- [] בוצעה הערכת סיכונים ראשונית מלאה
- [] נבנתה תכנית טיפול בסיכונים
- [] הופקה ואושרה הצהרת ההתאמה (SoA)
- [] הוגדרו יעדי אבטחת מידע מדידים

תמיכה ותפעול

- [] הוקצו משאבים מספקים (תקציב, כוח אדם, כלים)
- [] בוצעה הערכת מיומנות ותוכניות הכשרה לתפקידי מפתח
- [] בוצעה הדרכת מודעות לכלל העובדים
- [] הוגדר תהליך תקשורת פנימי/חיצוני בנושאי אבטחת מידע

- [] מבנה מסמכים מתועד (מדיניות → נהלים → הנחיות → רשומות) בשליטה
- [] הבקורות שנבחרו ב-SoA יושמו בפועל (טכנית וניהולית)
- [] קיים תהליך מוגדר לדיווח וטיפול באירועי אבטחת מידע

הערכה ושיפור

- [] הוגדרו מדדים (KPIs) ותהליך איסוף ודיווח שוטף
- [] בוצעה ביקורת פנימית מלאה, בלתי-תלויה, עם דו"ח ממצאים
- [] בוצע סקר הנהלה עם כל הקלטים הנדרשים (9.3)
- [] טופלו אי-התאמות שעלו, כולל ניתוח סיבה שורשית
- [] נאסף לפחות 2-3 חודשי עדות תפעולית לפני ביקורת הסמכה

הסמכה

- [] נבחר גוף הסמכה מוכר (Accredited, חבר IAF)
- [] עבר בהצלחה Stage 1 Audit וטופלו הממצאים
- [] עבר בהצלחה Stage 2 Audit וטופלו הממצאים
- [] התקבלה תעודת הסמכה
- [] נקבעה תכנית ביקורות מעקב שנתיות ותהליך שימור מתמשך

אחזקה שוטפת (Continuous)

- [] מחזור PDCA פעיל: הערכת סיכונים מתעדכנת, SoA מתעדכן בעת שינויים
- [] תכנית ביקורות פנימיות שנתיות מתקיימת בפועל
- [] סקרי הנהלה מתקיימים לפי התכנון
- [] מעקב שוטף אחר עדכוני התקן (כמו המעבר 2013→2022, ראו פרק 7)

8.3 דוגמאות לשאלות ריאיון ביקורת

שאלות שמבקר (פנימי או חיצוני) עשוי לשאול בעלי תפקידים שונים — שימושי גם כהכנה עצמית לפני ביקורת אמיתית:

לעובד מן השורה: - "מה עושים אם קיבלת מייל חשוד?" - "איפה אפשר למצוא את מדיניות אבטחת המידע של הארגון?" - "מתי הייתה ההדרכה האחרונה שעברת בנושא?"

לבעל נכס/מנהל מערכת: - "איך ידעת אילו בקורות ליישם על המערכת הזו?" (מצפים לתשובה שמפנה להערכת סיכונים ול-SoA, לא "כי ככה תמיד עשינו") - "מתי בפעם האחרונה סקרת את זכויות הגישה למערכת?"

ל-CISO/מוביל ה-ISMS: - "הראה לי ממצא מביקורת פנימית אחרונה, ומה נעשה איתו." - "איך אתה יודע שהביקורות שנבחרו ב-SoA מספיקות מול הסיכונים שזוהו?" - "מה קרה בסקר ההנהלה האחרון — אילו החלטות התקבלו?"

להנהלה בכירה: - "איך אתה עוקב אחרי מצב אבטחת המידע בארגון?" - "מה הייתה הפעם האחרונה שהקצית משאב/תקציב נוסף לנושא אבטחת מידע, ולמה?"

תשובה מהוססת, לא-עקבית בין בעלי תפקידים שונים, או תשובה שמסתמכת על "זה תמיד ככה" בלי הפניה לתהליך מתועד — הן דגלים אדומים קלאסיים שמבקרים מנוסים יודעים לזהות.

לחזרה לתוכן העניינים המלא — ראו [README.md](#). המשך ישיר: [פרק 9](#) — [בדיקות חדירה \(Pentesting\) ו-ISO/IEC 27001](#).

פרק 9: בדיקות חדירה (Pentesting) ו-ISO/IEC 27001

9.1 מה בדיוק דורש התקן – ומה הוא לא דורש

זו נקודה שמבלבלת ארגונים רבים: **ISO/IEC 27001 לא מכיל את המילה "penetration test" כדרישה מפורשת**. אין סעיף שאומר "יש לבצע בדיקת חדירה פעם בשנה". במקום זאת, התקן דורש **תוצאות** – ניהול חולשות טכניות אפקטיבי, ראייה שבקורות עובדות, והערכת סיכונים מבוססת-ראיות – ומשאיר לארגון לבחור באיזה אמצעי הוא מגיע לתוצאות האלה. בפועל, בדיקת חדירה היא **אחת הטכניקות המרכזיות והנפוצות ביותר** להשגת התוצאות שהבקורות הבאות דורשות:

בקרת נספח A	איך pentest משרת אותה
A.8.8 – ניהול חולשות טכניות	pentest הוא מקור מרכזי לזיהוי חולשות בסביבת ההפעלה בפועל, לא רק סריקה אוטומטית
A.8.29 – בדיקות אבטחה בפיתוח ובקבלה	pentest ליישומים/גרסאות לפני עלייה לפרודקשן
A.5.36 – סקירת עמידה במדיניות ובתקנים טכניים	pentest מספק עדות אובייקטיבית לפער בין מדיניות למציאות
A.5.7 – מודיעין איומים	ממצאי pentest (וטכניקות תוקפים אמיתיות) מזינים את הבנת האיום הרלוונטי לארגון
A.8.16 – פעילויות ניטור	pentest בודק אם הניטור בפועל מזהה פעילות תוקף (לעיתים כ-Purple/Red Team)
8.2 / 6.1.2 – הערכת סיכונים	ממצאי pentest הם קלט קונקרטי, לא תיאורטי, להערכת סיכונים
10.2 – אי-התאמה ופעולה מתקנת	ממצא pentest חמור שלא נסגר = לרוב אי-התאמה של ממש בביקורת

המשמעות המעשית: מבקר הסמכה (Certification Auditor) לא ישאל "תראו לי את דוח בדיקת החדירה" כדרישה פורמלית – אבל כן ישאל "איך אתם יודעים שבקורות ה-A.8.8/A.8.29 שלכם אפקטיביות?", ותשובה משכנעת כמעט תמיד כוללת pentest תקופתי עם ראייה לטיפול בממצאים.

9.2 סוגי בדיקות – טרמינולוגיה חיונית

בלבול נפוץ הוא זיהוי מוטעה בין כלים/שירותים שנשמעים דומה אך שונים מהותית באופיים, בעלות, ובערך שהם מספקים:

סוג	מה זה	עומק	תדירות טיפוסית
סריקת חולשות (Vulnerability Scan)	כלי אוטומטי (Nessus, Qualys, OpenVAS) מזהה חולשות ידועות	רדוד, ללא ניצול בפועל	שוטף (שבועי/חודשי)
בדיקת חדירה (Penetration Test)	בודק אנושי מנסה לנצל בפועל חולשות, מדמה תוקף עם יעד מוגדר	עמוק, ידני+כלים	שנתי / אחרי שינוי מהותי
Red Team	סימולציה ארוכת-טווח ומחמיקה של תוקף מתוחכם, לרוב ללא ידיעת צוות ה-SOC (מבחן Purple אם בשיתוף)	עמוק מאוד, ריאליסטי, ממוקד-מטרה לא ממוקד-כיסוי	מתקדם, לא לכל ארגון
Bug Bounty	תגמול לחוקרים חיצוניים תמורת דיווח חולשות, כיסוי מתמשך	משתנה מאוד	מתמשך
סקירת קוד/ארכיטקטורה (Code/Architecture Review)	בדיקה סטטית של קוד/עיצוב, לא של המערכת הרצה	עמוק בשכבת הקוד	שלב פיתוח, לפי SDLC

בתוך "בדיקת חדירה" עצמה יש עוד ציר סיווג:

- **Black box** — הבודק מקבל רק מטרה (למשל דומיין), ללא מידע פנימי — מדמה תוקף חיצוני חסר-ידע מוקדם.
 - **Grey box** — הבודק מקבל מידע חלקי (למשל חשבון משתמש רגיל) — מדמה תוקף עם דריסת רגל התחלתית (עובד זדוני, לקוח, שותף).
 - **White box** — הבודק מקבל גישה מלאה (קוד מקור, ארכיטקטורה, אישורים) — הכי יעיל למציאת חולשות עומק, פחות מדמה תוקף אמיתי.
- וכן לפי **וקטור**: יישומי Web (OWASP), רשת/תשתית (חיצונית/פנימית), אלחוט, הנדסה חברתית (פשיג, Vishing), פיזי (ניסיון כניסה פיזית), ענן (AWS/Azure/GCP misconfig), ומובייל.

9.3 מתודולוגיות מוכרות בעולם

בעת הזמנת/ביצוע pentest, שווה להסכים מראש על מתודולוגיה מוכרת — זה גם מבטיח כיסוי עקבי וגם מקל על מבקר הסמכה להעריך את איכות התהליך:

- **PTES — Penetration Testing Execution Standard**: מסגרת מקיפה לכל שלבי המבדק (Pre-engagement, Intelligence Gathering, Threat Modeling, Vulnerability Analysis, Exploitation, Post-Exploitation, Reporting).
- **OWASP Testing Guide / OWASP Top 10**: הסטנדרט המוביל לבדיקת יישומי Web — רשימת קטגוריות החולשות הנפוצות ביותר (Injection, Broken Access Control, Cryptographic Failures וכו') ומתודולוגיית בדיקה מפורטת לכל אחת.
- **NIST SP 800-115** — "Technical Guide to Information Security Testing and Assessment": מסגרת אמריקאית רשמית, שימושית כשצריך ליישר קו עם דרישות רגולטוריות אמריקאיות מקבילות.

- **OSSTMM — Open Source Security Testing Methodology Manual**: מתודולוגיה מדידה-מבוססת (Risk Assessment Values) לבדיקות מקיפות יותר מווב לבד.
- **MITRE ATT&CK** — לא מתודולוגיית pentest קלאסית אלא מסגרת טקטיקות ותת-טכניקות אמיתיות של תוקפים; משמשת בעיקר לתכנון תרחישי Red Team ולמידת יכולת גילוי (Detection Coverage).
- **+CompTIA PenTest** — לא מתודולוגיה עצמאית אלא תוכנית הסמכה (ראו גם 9.15) שמסגרת את התהליך בחמישה תחומי ידע (domains) שמשמשים בפועל גם כרשימת-מסגרת נוחה לתכנון עבודה: **(1) Planning and Scoping** — הרשאה, ROE, דרישות תאימות (חופף לסעיף 9.7 ולפרק 5); **(2) Information Gathering and Vulnerability Scanning** — recon וסריקה (חופף ל-9.4 שלבים 1-2); **(3) Attacks and Exploits** — ניצול בפועל בגבולות ה-ROE (חופף ל-9.4 שלב 3, ולפרק 10); **(4) Reporting and Communication** — דיווח ותקשורת ממצאים (חופף ל-9.4 שלב 4 ול-9.12); **(5) Tools and Code Analysis** — הכרת קטגוריות הכלים (חופף ל-9.13) וניתוח קוד/סקריפטים בסיסי. יתרון המסגרת הזו: מספקת "רשימת מכולת" תמציתית שקל למפות אליה שלבי עבודה בפועל, גם למי שלא ניגש להסמכה עצמה.

9.4 מ-Scoping ועד Retest — מיפוי לתהליך ה-ISMS

- מחזור עבודה טיפוסי לבדיקת חדירה, ואיך כל שלב "נתפר" לתוך תהליך ה-ISMS:
1. **הרשאה כתובה ו-Rules of Engagement (ROE)** — לפני כל פעולה. ראו את `pentest-milatova/scope.md` ו-`pentest-booknet/scope.md` בריפו זה כדוגמאות ממשיות למסמך `scope`: הגדרת מטרה, מה מותר/אסור (ללא DoS, ללא כתיבה/מחיקה, ללא ניסיונות עקיפת אימות ללא אישור מפורש), וחשיפת הרשאה כתובה שנשמרת מחוץ למאגר הקוד אם היא רגישה. זה תואם ישירות לדרישות A.5.31 (דרישות משפטיות/חוזיות) ולעקרון "בדוק רק את מה שהורשית לבדוק".
 2. **איסוף מודיעין וסריקה** — recon פסיבי/אקטיבי, זיהוי טכנולוגיות ונתיבי חשיפה.
 3. **זיהוי וניצול חולשות** — בהתאם לגבולות ה-ROE (לדוגמה, `pentest-milatova` הוגבל ל-GET בלבד, ללא ניצול/brute-force בפועל — מה שהופך אותו ל"סקר פסיבי" יותר מ"מבדק חדירה מלא", והבחנה זו **חייבת** להופיע בדוח כדי שלא יוצג רושם שווא של עומק הבדיקה).
 4. **דיווח (Reporting)** — כל ממצא מתועד עם: תיאור, חומרה, השפעה, שורש הבעיה, עדות (evidence), והמלצת תיקון. ראו את `pentest-milatova/findings/002-privileged-user-data-exposure.md` כדוגמה לדיווח מלא (Endpoint, Description, Root cause), `Impact, Evidence, Remediation` — מבנה זה כמעט זהה למה שמבקר ISO יצפה לראות ברשומת סיכונים.
 5. **תרגום ממצאים לסיכון וטיפול** — כאן ה-pentest "נכנס" רשמית ל-ISMS: כל ממצא ממופה לחולשה טכנית (לרוב CWE), מקבל ניקוד סיכון, ומוזן לתכנית טיפול בסיכונים (6.1.3) עם בעל סיכון ולוח זמנים. ראו סעיף 9.6 למטה — דוגמה מלאה שממירה את שני הממצאים האמיתיים מ-`pentest-milatova` לרשומת סיכונים בעזרת הכלי הנלווה לספר.

6. **תיקון (Remediation)** — פעולה מתקנת (10.2), עם אחראי ולוח זמנים לפי רמת הסיכון (למשל: Critical/High — ימים בודדים עד שבועות; Medium — עד רבעון; Low — מחזור התחזוקה הבא).

7. **בדיקה חוזרת (Retest)** — אימות שהתיקון אכן סגר את הפער — עדות קריטית להוכחת אפקטיביות הפעולה המתקנת (חלק מ-10.2), ולעיתים קרובות הדבר הראשון שמבקר יבקש לראות ("הראו לי לא רק את הממצא — הראו לי שהוא נסגר").

9.5 תדירות ו"טריגרים" ליישום pentest

התקן לא קובע תדירות מספרית, אך הפרקטיקה המקובלת (ומה שרוב גופי ההסמכה יצפו לראות כ"בסיס סביר" של ניהול סיכונים) היא:

- **לפחות אחת לשנה** לכל מערכת/שירות בהיקף ה-ISMS שחשופ לאינטרנט או מעבד מידע רגיש.
- **לפני עלייה משמעותית לפרודקשן** — פיצ'ר מהותי חדש, שינוי ארכיטקטורה, מיזוג/רכישה של מערכת.
- **לאחר שינוי משמעותי בתשתית** — מעבר ענן, שדרוג גרסת פלטפורמה מהותית.
- **דרישה חוזית/רגולטורית** — לקוחות ארגוניים, PCI DSS (דורש במפורש pentest שנתי + אחרי שינוי מהותי), חוזי SaaS B2B.
- **כחלק מהכנה להסמכה ראשונית** — לפני Stage 2 (פרק 5), רצוי pentest שמכסה את היקף ה-ISMS המוצהר, כדי שממצאים לא יתגלו לראשונה על ידי המבקר.

9.6 דוגמה עובדת: מ-Pentest אמיתי לרשומת סיכונים

הריפו הזה כולל שני תיקי engagement אמיתיים — [pentest-milatova/](#) ו-[pentest-booknet/](#) — עם `scope.md`, `REPORT.md` וממצאים מתועדים. נשתמש בממצאי [pentest-milatova](#) כדוגמה מלאה למעבר מדוח pentest לרשומת סיכונים לפי מתודולוגיית ISO/IEC 27005 (פרק 4, סעיף 6.3 בפרק 6):

ממצא מקורי	חומרה בדוח	CWE ממופה
חשיפת שדות <code>context-edit</code> <code>is_super_admin</code> (וכו') ל- <code>anon</code> דרך <code>wp/v2/users</code>	High	CWE-862 — Missing Authorization (חסר <code>permission_callback</code>)
חשיפת <code>username/slug</code> דרך <code>wp/v2/users</code>	Low	CWE-200 — Exposure of Sensitive Information

```
python3 iso27001-book/tools/cwe_risk_calculator.py report \
--input iso27001-book/tools/pentest_case_study_findings.json
```

תוצאה:

cwe_id	name				likelihood	impact_max
asset						
score	risk_level					
-----+-----						
+-----+-----+-----						
CWE-862	Missing Authorization				milatova.org.il - wp-json/	
wp/v2/users (privileged edit-context fields)	4	4	16	גבוה (High)		
CWE-200	Exposure of Sensitive Information to an Unauthorized Actor				milatova.org.il - wp-json/	
wp/v2/users (username/slug enumeration)	4	1	4	נמוך (Low)		

שני שיעורים חשובים מהדוגמה הזו:

1. **הממצא הראשון (CWE-862) מקבל אוטומטית מתוך טבלת ברירת המחדל של הכלי בדיוק את אותה רמת חומרה (High) שהמבקר האנושי קבע** — עדות לכך שממצא "פרצת הרשאות בממשק ציבורי" הוא מהותית High ברוב ההקשרים, בלי צורך בכיול מיוחד.
2. **הממצא השני (CWE-200) דרש דריסה ידנית (impact_c=1)** במקום ברירת המחדל (4) כדי לשקף שחשיפת username ב-WordPress היא התנהגות מוכרת ולא-רגישה בפני עצמה — בלי הדריסה, הכלי היה מציג "High" באופן מוטעה. זו בדיוק הנקודה שהודגשה כבר בפרק 6 ובקובץ `tools/README.md`: **טבלת ה-CWE הבסיסית היא נקודת פתיחה להמחשה, לא תחליף לשיקול דעת אנושי בהקשר הספציפי של הממצא.**

9.7 עקרונות אתיים ומשפטיים — לא משא ומתן

- **אין לבצע כל פעולה פוגענית (exploitation, brute-force, DoS) ללא הרשאה כתובה ומפורשת** מבעל המערכת/הארגון, עם היקף מוגדר בכתב (מסמך scope/ROE חתום).
- **תיעוד ROE חייב לכלול:** מטרות מוגדרות, טווח IP/דומיינים מותרים, חלון זמן, אנשי קשר לעצירת חירום (Emergency Stop), מה אסור באופן מפורש (למשל: לא לבדוק זרימות תשלום אמיתיות, לא להשתמש בפרטי לקוחות אמיתיים).
- **הפרדת סביבות** — עדויות רגישות (תגובות גולמיות עם PII, אישורים שהתגלו) יש לשמור בנפרד, עם גישה מוגבלת, ולשקול לא לשמור אותן במאגר קוד משותף (ראו הערת ה-scope בקובצי pentest-milatova/pentest-booknet לגבי שמירת הסכם החתום מחוץ למאגר).
- **גילוי אחראי (Responsible Disclosure)** — כשמזהה חולשה בתלות צד-שלישי (ספרייה, פלאגין) שלא בבעלות הארגון עצמו, יש לדווח לספק בערוץ מוגדר (security.txt, תוכנית bug bounty) לפני חשיפה ציבורית.
- **אסור לבלבל בין "אין הרשאה מפורשת לבדיקה אקטיבית" לבין "מותר הכול כי המידע ציבורי"** — גם מידע שנגיש טכנית (כמו endpoint ציבורי) עשוי להיות מחוץ להיקף אם ה-ROE לא הרשה זאת במפורש.

9.8 טעויות נפוצות בשילוב ISMS ו-Pentest

- **"עשינו pentest פעם, סימנו וי"** — pentest חד-פעמי בלי מחזוריות מקביל לביקורת פנימית חד-פעמית: לא עומד ברוח סעיף 9 (הערכת ביצועים מתמשכת).

- **דוח מרשים שנשאר במגירה** — ממצאים ללא בעל סיכון, ללא תאריך יעד, ללא Retest, הם ראייה נגד הארגון בביקורת ("זיהיתם את הבעיה וגם לא טיפלתם בה" גרוע יותר מ"לא בדקתם").
- **היקף (Scope) לא תואם ל-ISMS Scope** — pentest שבדק רק חלק קטן מהמערכות בתוך היקף ה-ISMS המוצהר משאיר "עיוורון" שהמבקר עלול לתפוס.
- **בלבול בין Pentest ל-Vulnerability Scan** — הצגת פלט כלי אוטומטי כ"בדיקת חדירה" בפני מבקר או הנהלה, בלי בדיקה/ניצול אנושי בפועל.
- **דירוג חומרה לא-מכיל** — הסתמכות עיוורת על ברירת מחדל גנרית (כמו טבלת CWE הבסיסית בכלי הנלווה) בלי לשקלל הקשר ארגוני אמיתי — ראו סעיף 9.6 לעיל.

9.9 סדר עדיפויות בזמן מוגבל

בדיקת חדירה כמעט תמיד מתבצעת תחת אילוץ זמן (חלון engagement קצוב), ולכן השאלה הפרקטית ביותר אינה "מה לבדוק" (התשובה — הכול שבתקציב הזמן מרשה) אלא **באיזה סדר**. העיקרון המנחה: לתעדף לא לפי "מה הכי מסובך טכנית" אלא לפי מה שנותן לתוקף את הדריסה הגדולה ביותר, הכי מהר — כלומר בדיוק אותו עיקרון $Likelihood \times Impact$ שהכלי הנלווה לספר מיישם (פרק 6.3, 9.6). סדר עדיפויות מומלץ, מהדחוף ביותר:

1. **בקרת גישה שבורה / הרשאות חסרות (Broken Access Control)** — נקודות קצה מנהליות או API-ים שאמורים לדרוש הרשאה ולא בודקים זאת בפועל. זוהי קטגוריית החולשה מספר 1 ב-OWASP Top 10, וזה בדיוק הממצא ה-High מ-pentest-milatova (CWE-862, סעיף 9.6) — חשיפת "מי הוא super-admin" נותנת לתוקף מפת דרכים מיידית עוד לפני שניסה לפרוץ בפועל.
2. **הזרקות (Injection — SQLi, Command Injection)** — פחות שכיחות מבעבר, אך כשקיימות ההשפעה כמעט תמיד קריטית (CWE-89/CWE-78 בטבלת הכלי: impact 4-5 בכל שלושת צירי ה-CIA).
3. **סודות חשופים / קרדנציאלים קשיחים בקוד** (CWE-798/259/522) — לרוב הדרך המהירה ביותר להשתלטות מלאה, וזולה יחסית לאיתור (חיפוש בקוד, בהיסטוריית git, בקבצי קונפיגורציה) — שווה לבדוק מוקדם ב-engagement.
4. **טביעת אצבע גרסאות / CVE ידועים** — ראו את ממצא ה-Info ב-pentest-milatova/REPORT.md: חשיפת גרסת Yoast SEO ב-HTML comment. זו בדיקה זולה שיכולה לקצר משמעותית את שאר הבדיקה אם הגרסה חשופה ל-CVE ציבורי.
5. **חשיפת מידע רגיש / הצפנה חסרה** (CWE-200/CWE-311/CWE-319) — פחות "מפוצץ" אך נפוץ מאוד, ולעיתים מזין את השלבים הקודמים (למשל חשיפת username מזינה ניחוש קרדנציאלים, ראו הממצא ה-Low באותה דוגמה).

כלל אצבע לתעדוף: מה שחשוף לאינטרנט + לא דורש אימות מוקדם + מעניק הרשאות/מידע רחב-היקף מנצח כל חולשה תיאורטית שדורשת שרשרת תנאים ספציפית לניצול — בדיוק ההבדל בין ציון 16 ל-4 בדוגמה שבסעיף 9.6.

CVSS 9.10 – ניקוד חומרה משלים ל-CWE

בעוד ש-CWE מזהה את סוג החולשה ("מה זה"), **CVSS – Common Vulnerability Scoring System** מנקד חומרה ספציפית ("כמה זה חמור, כאן, במקרה הזה") – ולכן שני הכלים משלימים זה את זה, לא מתחרים. דוחות pentest מקצועיים ורוב מסדי ה-CVE הציבוריים מדווחים ציון CVSS (גרסה 3.1 נכון להיום) לצד ה-CWE.

מדדי הבסיס העיקריים ב-CVSS v3.1 (Base Metrics):

מדד	משמעות
AV – Attack Vector	דרך הניצול: Network / Adjacent / Local / Physical
AC – Attack Complexity	כמה קשה לנצל בפועל: Low / High
PR – Privileges Required	אילו הרשאות נדרשות מראש לתוקף: None / Low / High
UI – User Interaction	האם נדרשת פעולת משתמש (למשל קליק על קישור): None / Required
S – Scope	האם הניצול "בורח" מגבולות הרכיב הפגיע להשפיע על רכיבים אחרים
C/I/A	השפעה על סודיות/שלמות/זמינות: None / Low / High

הציון הסופי (0.0-10.0) מתורגם לרמות: **Medium (7.0-8.9), High (8.0-9.0), Critical (9.0-10.0)** – סולם דומה במהות לסולם ה-Low/Medium/High/Critical של הכלי הנלווה לספר, אך CVSS מחושב מתוך 8 מדדים פורמליים ומדויקים יותר, בעוד מודל הספר (Likelihood × Impact) הוא פשוט מכון-ISO/IEC 27005 שקל יותר להטמיע ידנית. **בפועל:** דוח pentest טוב מצרף ציון CVSS לכל ממצא ספציפי (לתקשורת חיצונית/עם ספקים), **וגם** מתרגם אותו לרמת סיכון ארגונית לפי הקשר (כפי שנעשה בסעיף 9.6) – כי ציון CVSS "גולמי" לא תמיד משקף את ההשפעה העסקית האמיתית על הארגון הספציפי (נכס לא-קריטי עם CVSS 9.0 עשוי להיות סיכון ארגוני נמוך יותר מנכס קריטי עם CVSS 6.5).

OWASP Top 10 9.11 – מיפוי לקטגוריות CWE

OWASP Top 10 היא רשימת עשר קטגוריות הסיכון הנפוצות והמשמעותיות ביותר ביישומי Web, מתעדכנת מעת לעת על ידי קהילת OWASP (המהדורה הנוכחית: 2021). הטבלה הבאה ממפה כל קטגוריה לדוגמאות CWE – כולל כאלה שכבר מופיעים בטבלת הבסיס של הכלי הנלווה לספר:

#	קטגוריית OWASP Top 10 (2021)	דוגמאות CWE נפוצות
A01	Broken Access Control	CWE-862, CWE-863, CWE-284
A02	Cryptographic Failures	CWE-311, CWE-319, CWE-327, CWE-330
A03	Injection	CWE-89, CWE-78, CWE-79, CWE-94
A04	Insecure Design	(רחב – פגמים ארכיטקטוניים, לא bug נקודתי)
A05	Security Misconfiguration	CWE-16, CWE-611, CWE-1188

#	קטגוריית (2021) OWASP Top 10	דוגמאות CWE נפוצות
A06	Vulnerable and Outdated Components	(זיהוי גרסאות/CVE — ראו 9.4 שלב 4)
A07	Identification and Authentication Failures	CWE-287, CWE-306, CWE-522
A08	Software and Data Integrity Failures	CWE-502, CWE-829
A09	Security Logging and Monitoring Failures	(חופף ל-8.16–8.15 A בפרק 3)
A10	Server-Side Request Forgery (SSRF)	CWE-918

טבלה זו שימושית לתכנון כיסוי בדיקה: מבדק חדירה ליישום Web שלא מכסה לפחות את עשר הקטגוריות האלה (או שמסביר במפורש למה קטגוריה מסוימת לא רלוונטית ליישום הנבדק) מפספס את הבסיס המוסכם בתעשייה.

9.12 תבנית לדוח pentest (מבנה כללי)

מבנה דוח מומלץ, מיושר עם מה שנראה בפועל ב- `pentest-milatova/REPORT.md` (פרק 9.6) ועם מתודולוגיית PTES (9.3):

1. Executive Summary (תקציר מנהלים)
 - היקף, תאריכים, מספר ממצאים לפי חומרה, מסקנה כללית על מצב האבטחה
2. Engagement (ראו ROE, היקף, הרשאה, מגבלות – `scope.md`)
3. מתודולוגיה – אילו שלבים/מתודולוגיה הופעלו (9.3–9.4)
4. ממצאים, כל אחד עם:
 - (תרגום לרמת סיכון ארגונית + CVSS) כותרת + חומרה
 - Endpoint/Asset מדויק
 - תיאור טכני
 - ממופה CWE
 - (Root Cause) שורש הבעיה
 - השפעה עסקית (לא רק טכנית)
 - (מצולם/מתועד, evidence) עדות
 - המלצת תיקון קונקרטי (לא "תקנו את זה" אלא צעד-אחר-צעד)
5. סיכום טבלת ממצאים (# | חומרה | כותרת | סטטוס)
6. (Out of scope) המלצות להמשך / בדיקות שלא בוצעו
7. נספח: פרטי סביבת הבדיקה, כלים ששימשו, לוג בקשות (אם רלוונטי)

שימו לב: **סעיף 6 (מה לא נבדק)** קריטי לא פחות מהממצאים עצמם — דוח שמשתמע ממנו "בדקנו הכול ולא מצאנו כלום" בלי לפרט את המגבלות (כמו ב-ROE של `pentest-milatova`, שהוגבל ל-GET בלבד) מטעה את קוראיו לגבי רמת הביטחון שמותר להסיק ממנו.

9.13 קטגוריות כלים נפוצות (סקירה, לא הדרכת שימוש)

לצורך אוריינטציה בלבד — קטגוריות הכלים הנפוצות בעבודת pentest מורשית, ללא הבחנה בין כלי ספציפי (רבים מהם קוד-פתוח ומשמשים גם צוותי הגנה לבדיקה עצמית):

שלב	קטגוריית כלי	דוגמאות מוכרות
Recon/סריקה	סורקי פורטים/שירותים	Nmap
Recon/Web	סורקי תוכן/דירים	Gobuster, ffuf
בדיקת יישומי Web	Proxy לבדיקה ידנית+אוטומטית	Burp Suite, OWASP ZAP
בדיקת חולשות ידועות	סורקי חולשות	Nessus, OpenVAS
ניצול (במסגרת ROE בלבד)	מסגרות ניצול	Metasploit
ניתוח קובץ חשוד שהתגלה (webshell/stager)	אנליטיקה סטטית	Semgrep, YARA + PHP-Malware-Finder, Bandit (Python), ESLint-security (JS/TS) — ראו tools/README.md
דיווח	ניהול ממצאים/דוחות	כלי GRC/ניהול פרויקט, תבניות Markdown כמו בריפ הזה

כל כלי מהרשימה הזו הוא **דו-שימושי (dual-use)** — אותו כלי בדיוק שמשמש בודק מורשה משמש גם תוקף. ההבדל היחיד, תמיד, הוא **הרשאה כתובה ו-ROE מוגדר** (סעיף 9.7) — לא הכלי עצמו.

9.14 פרוטוקול בדיקה ייעודי לכל CWE

הטבלאות בפרקים 6 ו-9 השתמשו עד כה ב-CWE כדי **לנקד** ממצא (Likelihood × Impact). אבל לפני שיש בכלל ממצא לנקד — צריך לדעת **איך בודקים** כל חולשה בפועל. הטבלה הבאה מוסיפה את השכבה הזו: לכל אחד מ-41 ה-CWEים בטבלת הבסיס של `cwe_risk_calculator.py` (זהה בדיוק לרשימה שבכלי — ראו `test_protocol` בקוד המקור), פרוטוקול הבדיקה הקונקרטי שבודק חדירה מורשה מפעיל כדי לאמת (או לשלול) את קיום החולשה. כל הטכניקות להלן מיועדות **אך ורק** לבדיקה מורשית תחת ROE מוגדר (סעיף 9.7) — לא כהדרכת ניצול.

CWE	שם	פרוטוקול בדיקה
CWE-79	Cross-Site Scripting (XSS)	הזרקת payloads (<code><script></code> , event handlers , מקודדים) לכל פרמטר קלט המוצג בדף; אימות הרצה בפועל בהקשר ה-DOM/attribute/JS (reflected/stored/DOM-based)
CWE-89	SQL Injection	הזרקת payloads בוליאניים/מבוססי-זמן/מבוססי-שגיאה (OR ' , '1='1 , SLEEP(5)) בכל פרמטר; ניתוח הבדלי תגובה; אימות עם חילוץ מבוקר (UNION-based) תחת ROE בלבד
CWE-78	OS Command Injection	הזרקת מפרידי פקודות (; , pipe , && , ` \$ ( ) `) לפרמטרים שעשויים לעבור ל-shell; זיהוי עיכוב מכוון (sleep) כאינדיקציה עיוורת; אימות בסביבת בדיקה מבוקרת
CWE-94	Code Injection	הזרקת payloads לשפת תבנית/SSTI eval , למשל { {7*7} } ; זיהוי חישוב בפועל בתגובה כאינדיקציה לביצוע קוד
CWE-22	Path Traversal	

CWE	שם	פרוטוקול בדיקה
		בקשת נתיבים עם <code>../</code> (מקודד/לא-מקודד) לקבצים ידועים (<code>/etc/passwd</code> , <code>web.config</code>); בדיקת עקיפת סינון עם <code>null-byte</code> / <code>double-encoding</code>
CWE-352	CSRF	בניית טופס/בקשה חוצה-מקור ללא טוקן CSRF תקף; אימות שהפעולה מתבצעת בהצלחה כשמשמש מחובר פותח עמוד חיצוני זדוני
CWE-434	Unrestricted File Upload	העלאת קבצים עם סיומות מבוצעות (<code>.php</code> / <code>.jsp</code> / <code>.asp</code>) תוך מיפולציית <code>Content-Type/magic bytes/double extension</code> ; אימות שהקובץ נגיש ומופעל בשרת
CWE-502	Insecure Deserialization	איתור נקודות קצה שמקבלות אובייקטים מסודרים (<code>java/PHP</code> / <code>Python pickle</code> וכו') והזרקת <code>gadget chains</code> ידועים (למשל <code>ysoserial</code>) במסגרת ROE בלבד
CWE-611	XXE	שליחת מסמכי XML עם <code>DOCTYPE/ENTITY</code> חיצוני המצביע לקובץ מקומי או URL פנימי; בדיקת אקספילטרציה חוץ-לפס (OOB)
CWE-287	Improper Authentication	בדיקת עקיפה/החלשה של זרימת האימות: ניחוש/brute-force מבוקר לפי ROE, עקיפת <code>session fixation</code> , <code>2FA</code> , נתיבי אימות חלופיים
CWE-306	Missing Authentication for Critical Function	גישה ישירה ל-endpoint רגיש ללא כל טוקן/session; השוואה מול קריאה מאומתת לאותה פעולה
CWE-862	Missing Authorization	גישה ל-endpoint עם session ברמת הרשאה נמוכה (או ללא session כלל) ובדיקה אם מידע/פעולה מוגבלים מוחזרים בכל זאת — ראו דוגמת <code>milatova</code> ב-9.6
CWE-863	Incorrect Authorization (IDOR)	שינוי מזהי אובייקט בבקשות בין שני משתמשי-בדיקה שונים; בדיקה אם ניתן לגשת/לשנות נתוני המשתמש האחר
CWE-269	Improper Privilege Management	ניסיון הסלמת הרשאות אופקית/אנכית — שינוי <code>role/claim</code> בטוקן JWT או בפרמטר בקשה, ובדיקה אם השרת מכבד ערך מהלקוח
CWE-798	Hard-coded Credentials	חיפוש מחרוזות/מפתחות בקוד המקור, בהיסטוריית <code>git</code> , בקבצי תצורה שנחשפו (<code>.env</code> , <code>config.json</code>), ובבינארי/אפליקציית מובייל שפורקו
CWE-321	Hard-coded Cryptographic Key	חילוץ מפתחות קבועים מקוד/בינארי/אפליקציית מובייל; אימות אם הם משמשים בפועל בפרודקשן
CWE-522	Insufficiently Protected Credentials	בדיקת שידור/אחסון סיסמאות בטקסט גלוי (פרמטרי URL, עוגיות, לוגים); אימות מדיניות hashing (<code>bcrypt/argon2</code> מול <code>MD5/SHA1</code>)
CWE-259	Hard-coded Password	כמו CWE-798 אך ממוקד בסיסמה ספציפית — חיפוש בקוד/תצורה, ניסיון שימוש מול ממשקי ניהול/DB
CWE-200	Exposure of Sensitive Information	השוואת תגובות מאומתות מול לא-מאומתות לאותו endpoint; חיפוש שדות <code>edit-context</code> שדולפים בתגובה ציבורית — ראו 9.6
CWE-209		

CWE	שם	פרוטוקול בדיקה
	Sensitive Info in Error Messages	טריגור מכון לשגיאות (קלט לא תקין/סוג שגוי) ובדיקה אם stack/trace נתיבי קבצים/גרסאות נחשפים
CWE-732	Incorrect Permission Assignment	בדיקת הרשאות קבצים/משאבים (world-writable, ACL, דליים ציבוריים בענן) בסביבות שיש אליהן גישה
CWE-284	Improper Access Control	מיפוי שיטתי של כל ה-endpoints מול מטריצת הרשאות מוגדרת מראש, ובדיקת כל שילוב role/אפעולה
CWE-190	Integer Overflow	הזנת ערכים בקצוות טווח (MAX_INT, שליליים) בשדות מספריים; בדיקת עקיפת בדיקות תקציב/מכסה
CWE-125	Out-of-bounds Read	פאזינג קלטים בינאריים/פרוטוקולים עם ערכי אורך/אינדקס חריגים; ניטור קריסות/דליפת זיכרון ברכיבי native
CWE-787	Out-of-bounds Write	פאזינג דומה ל-CWE-125 עם מעקב קריסות המעידות על שכתוב זיכרון, לרוב עם sanitizers (ASAN)
CWE-416	Use After Free	בדיקה דינמית עם sanitizers (ASAN) בסביבת פיתוח/בדיקה ברכיבי native — לא מול פרודקשן
CWE-476	NULL Pointer Dereference	שליחת קלטים חסרים/ריקים (missing fields, null JSON) לנקודות קצה; בדיקת קריסה/DoS
CWE-311	Missing Encryption	בדיקת ערוצי העברה (HTTP מול HTTPS) ואחסון ללא הצפנה -at-rest; ניתוח תעבורה (mitmproxy/Wireshark)
CWE-327	Broken/Risky Crypto Algorithm	זיהוי אלגוריתמים חלשים (MD5/SHA1/DES/RC4) ב-TLS handshake, בקוד, או באחסון סיסמאות; כלים כמו /testssl.sh/sslyze
CWE-330	Insufficiently Random Values	דגימת מספר טוקנים/session-ids/reset-password ברצף ובדיקת דפוס/entropy נמוך; ניסיון חיזוי/שכפול
CWE-918	SSRF	הזנת URL פנימי/כתובת מטא-דאטה של ענן (169.254.169.254) בפרמטר שמבצע בקשת HTTP מהשרת; אימות SSRF עיוור עם callback חוץ-לפס
CWE-400	Uncontrolled Resource Consumption	שליחת בקשות/קלטים כבדים (payload גדול, ReDoS) בסביבת בדיקה מבוקרת ובאישור ROE מפורש — לרוב אסור בפרודקשן
CWE-668	Exposure of Resource to Wrong Sphere	בדיקת גישה חוצה-דיירים (multi-tenant) — ניסיון גישה למשאבי דייר אחר עם session תקף של הדייר הנוכחי
CWE-1188	Insecure Default Initialization	סקירת תצורת ברירת מחדל של רכיבים שהותקנו (DB, ענן, קונטיינרים) לאיתור admin/admin, פאנלים פתוחים
CWE-20	Improper Input Validation	פאזינג שיטתי של כל שדות הקלט עם ערכים חריגים/סוג שגוי/תווים מיוחדים, כבסיס לחולשות ממוקדות יותר
CWE-16	Security Misconfiguration	סקירת headers, קבצי חשיפה (robots.txt, .git/, .env), הרשאות ברירת מחדל — שלב recon ראשוני
CWE-295		

CWE	שם	פרוטוקול בדיקה
	Improper Certificate Validation	בדיקה שהלקוח מקבל תעודות לא-תקינות/self-signed/פגות-תוקף ללא שגיאה, בעזרת MITM proxy מבוקר
CWE-319	Cleartext Transmission	ניתוח תעבורת רשת (mitmproxy/Wireshark) לאיתור HTTP לא-מוצפן, פרוטוקולים legacy, נפילה ל-TLS ישן
CWE-346	Origin Validation Error (CORS)	בדיקת הגדרות CORS (* Access-Control-Allow-Origin: עם credentials, reflect של Origin) וקריאה חוצה-מקור עם עוגיות/טוקן
CWE-601	Open Redirect	הזנת URL חיצוני בפרמטרי redirect (?next= , ?return_url=) ואימות הפניה בפועל — שימושי כתמיכה בפישינג
CWE-1021	Clickjacking	בדיקת headers (X-Frame-Options, CSP frame-ancestors) והטמעת הדף ב-iframe בעמוד בדיקה לוודא חסימה

הטבלה הזו חיה גם בקוד, לא רק במסמך — כדי לא ליצור שני "מקורות אמת" שעלולים לסטות זה מזה עם הזמן. אפשר לקבל אותה ישירות מהכלי:

```
# ים-CWE-פרוטוקול בדיקה לכל 41 ה
python3 iso27001-book/tools/cwe_risk_calculator.py protocol

# ספציפי בלבד CWE-פרוטוקול בדיקה ל
python3 iso27001-book/tools/cwe_risk_calculator.py protocol --cwe CWE-862
```

9.15 תקינה ורגולציה בינלאומית בתחום בדיקות חדירה

מעבר למתודולוגיות (9.3) יש שכבה נוספת: **הסמכות אישיות** לבודקים, **הכרות ארגוניות** לחברות pentest, ו**רגולציה שמחייבת** ביצוע בדיקות במפורש. שכבה זו קובעת לא רק "איך בודקים" אלא "מי מוסמך לבדוק" ו"באיזו תדירות זה חובה על פי דין/רגולטור" — רלוונטי במיוחד כשבוחרים ספק pentest חיצוני או כשמדווחים לרגולטור/מבקר על עמידה בדרישה.

הסמכות אישיות לבודק (Practitioner Certifications)

הסמכה	גוף מנפיק	התמקדות
OSCP (Offensive Security Certified Professional)	OffSec	מבחן מעשי (hands-on) על סביבת מעבדה אמיתית — הסמכת הכניסה המוכרת ביותר לתחום ה-pentest המעשי
OSCE3 / OSWE / OSEP	OffSec	הסמכות מתקדמות: פיתוח קוד ניצול, אבטחת יישומי Web מתקדמת, עקיפת בקורות הגנה
CEH (Certified Ethical Hacker)	-EC Council	הסמכה תיאורטית-רחבה, לרוב נקודת כניסה לתחום, פחות מעשית מ-OSCP
GPEN / GWAPT / GXPN	/SANS GIAC	הסמכות מעשיות ממוקדות (בדיקות חדירה כלליות, יישומי Web, ניצול מתקדם)

הסמכה	גוף מנפיק	התמקדות
+CompTIA PenTest	CompTIA	הסמכה עצמאית-ספק, משלבת שאלות רב-ברירה וסימולציות מעשיות; מסגרת חמשת התחומים שלה (9.3) שימושית גם כתבנית תכנון עבודה
/CREST Registered Certified Tester	CREST	הסמכה בריטית שמוכרת גם באוסטרליה, סינגפור ועוד; נדרשת חוזית בחלק מהמכרזים הממשלתיים/פיננסיים

הכרות והסמכות ארגוניות (לחברת ה-pentest)

- **CREST** (Council of Registered Ethical Security Testers) – הסמכה ארגונית (לא רק אישית) שמוודאת שלחברה יש מתודולוגיה, ניהול ROE, והגנה על ממצאים רגישים ברמת החברה כולה – לא רק ברמת הבודק הבודד.
- **CHECK** – תוכנית ה-NCSC הבריטית לאישור ספקי pentest המורשים לעבוד מול גופי ממשל בריטיים.
- **TIBER-EU** (Threat Intelligence-Based Ethical Red Teaming) – מסגרת אירופאית (ביוזמת הבנק המרכזי האירופי) לביצוע Red Team מבוסס מודיעין איומים אמיתי כנגד מוסדות פיננסיים קריטיים; מדינות רבות אימצו גרסה לאומית (למשל TIBER-DE, TIBER-NL).
- **CBEST** – המסגרת הבריטית (Bank of England) שקדמה ל-TIBER-EU ומיישמת עיקרון דומה למגזר הפיננסי הבריטי.
- **DORA** (Digital Operational Resilience Act, האיחוד האירופי) – רגולציה (לא תקן וולונטרי) שמחייבת גופים פיננסיים משמעותיים לבצע **TLPT – Threat-Led Penetration Testing** תקופתי, בהשראת מודל TIBER-EU, ככלי אכיפה רגולטורי ולא רק המלצה.

תקני ISO תומכים לתהליך הדיווח על חולשות

שני תקנים ממשפחת ISO/IEC שלא הוזכרו עדיין (פרק 4) משלימים בדיוק את מה שקורה **אחרי** שממצא pentest מתגלה:

- **ISO/IEC 29147 – Vulnerability disclosure**: איך מדווחים באחריות על חולשה שהתגלתה לספק/ליצרן חיצוני (לא לארגון הנבדק עצמו) – כולל ערוצי דיווח, ציפיות לזמני תגובה, ותקשורת עם המדווח.
- **ISO/IEC 30111 – Vulnerability handling processes**: הצד המשלים – איך ספק/יצרן (Vendor) אמור **לקבל, לנתח, ולתקן** דיווח חולשה שהתקבל, כהליך פנימי סדור. רלוונטי לארגונים שמפתחים מוצר/שירות ומקבלים דיווחי חולשות מחוקרים חיצוניים (bug bounty, גילוי אחראי).

רגולציה שמחייבת pentest במפורש (לא רק ממליצה)

- **PCI DSS** (דרישות 11.3/11.4) – מחייב במפורש **pentest שנתי** לרשת ולאפליקציה, **בדיקת פילוח רשת (segmentation testing)** בתדירות גבוהה יותר, ו**pentest נוסף אחרי כל שינוי מהותי** בתשתית/אפליקציה שבהיקף PCI. זהו התקן עם הדרישה המפורשת והמוחשית ביותר לתדירות pentest מבין כל התקנים/הרגולציות שנסקרו בספר.

- **NIS2** (הדירקטיבה האירופית לאבטחת סייבר) — לא מנוסח כדרישת "pentest" מילולית, אך מחייב אמצעי ניהול סיכונים הכוללים בדיקות אבטחה תקופתיות לגופים בהיקפה — נאכף בפועל דרך רגולטורים לאומיים.
- **DORA** — כאמור, מחייב TLPT מפורש לגופים פיננסיים משמעותיים (בדרך כלל אחת לכמה שנים, בהתאם לפרופיל הסיכון של הארגון).
- **תקנות הגנת הפרטיות (אבטחת מידע), התשע"ז-2017 (ישראל)** — מחייבות בעלי מאגרי מידע ברמת אבטחה גבוהה לבצע **מבדקי חדירה (Penetration Test)** תקופתיים על ידי גורם מומחה, במרווחים קבועים שנקבעים לפי רמת האבטחה של המאגר — ראו את הנוסח המלא והמעודכן של התקנות לפני קביעת תדירות מדויקת בארגון ספציפי.
- **הוראת בנק ישראל (ניהול הגנת סייבר, הוראה 361)** — מחייבת תאגידים בנקאיים לשלב מבדקי חדירה וסימולציות תקיפה (כולל תרגילי Red Team) כחלק ממסגרת ניהול הגנת הסייבר השוטפת שלהם.

המסקנה המעשית: ארגון הבוחר ספק pentest צריך לבדוק לא רק את המתודולוגיה (9.3) אלא גם **אילו הסמכות אישיות/ארגוניות יש לספק** (מול מה שהחוזה/הרגולטור דורשים), **והאם תדירות הבדיקה שנקבעה בפועל** עומדת בדרישה הרגולטורית הרלוונטית לענף הארגון (לא רק בדרישת ISO/IEC 27001, שכאמור לא קובעת תדירות מספרית — ראו פרק 9.5).

9.16 התאמה למציאות הישראלית

- מעבר לתקנה הבינלאומית (9.15), ארגון הפועל בישראל נדרש להכיר גם את המסגרת המקומית — שלעיתים **מחמירה** יותר מהדרישה הבינלאומית המקבילה, ולעיתים פשוט מנוסחת אחרת:
- **תקן SI ISO/IEC 27001** — מכון התקנים הישראלי (מת"י) מאמץ את התקן הבינלאומי כתקן ישראלי רשמי (SI ISO/IEC 27001), במספור ובנוסח זהים למהות לתקן הבינלאומי שנסקר בפרקים 1-8. הסמכה ל-ISO/IEC 27001 "רגילה" (דרך גוף הסמכה בינלאומי מוכר) מוכרת כשוות-ערך לצורך רוב הדרישות הרגולטוריות/חוזיות בישראל — אין צורך בהסמכה "כפולה" נפרדת.
 - **מערך הסייבר הלאומי (הרשות הלאומית להגנת הסייבר)** — מפרסם הנחיות והמלצות (כולל מסמכי "מתודולוגיה להערכת סיכוני סייבר" ומדריכים ענפיים) שמומלץ להכיר בבניית תכנית ה-pentest הארגונית, גם אם אין דרישה מחייבת ישירה לכל ארגון.
 - **הרשות להגנת הפרטיות ותקנות הגנת הפרטיות (אבטחת מידע), התשע"ז-2017** (שכבר הוזכרו ב-9.15) — הן החקיקה הישראלית המרכזית שמחייבת בפועל מבדקי חדירה תקופתיים לבעלי מאגרי מידע ברמת אבטחה גבוהה, מכוח חוק הגנת הפרטיות, התשמ"א-1981. שימו לב למינוח: הרגולציה הישראלית משתמשת לרוב במונח "**מבדק חדירות**" או "**מבחן חדירה**" לצד "בדיקת חדירה"/"pentest" — אותו מושג, מינוחים מקבילים.
 - **בנק ישראל** — הוראת ניהול בנקאי תקין 361 (כבר הוזכרה ב-9.15) חלה על תאגידים בנקאיים; **רשות שוק ההון, ביטוח וחיסכון** מפרסמת הנחיות מקבילות לגופים מוסדיים (חברות ביטוח, קופות גמל, קרנות פנסיה) הכוללות דרישות ניהול סיכוני סייבר ובדיקות תקופתיות בעלות הגיון דומה.

- **רשות ניירות ערך (ISA)** — מפרסמת הנחיות גילוי ואבטחת סייבר לחברות ציבוריות, שרלוונטיות בעיקר לדיווח ולממשל (חופף לרעיון ISO/IEC 27014, פרק 4) ולעיתים משיקות לדרישת בדיקות תקופתיות כחלק מניהול הסיכונים המדווח.

- **שפה ומינוח:** מסמכי ROE/scope ודוחות pentest בישראל (ראו `pentest-milatova/scope.md` ו-`pentest-booknet/scope.md` ברפיו הזה) נכתבים לרוב באנגלית טכנית גם כשהתקשורת הארגונית עברית — שמרו על עקביות מינוח (CWE/CVSS/OWASP) גם בדוחות בעברית, כדי שהמידע יהיה ניתן להשוואה מול מאגרי מידע בינלאומיים (NVD, MITRE).

המלצה מעשית לארגון ישראלי: אם המאגר/המערכת מסווגים ברמת אבטחה גבוהה לפי תקנות הגנת הפרטיות — תדירות ה-pentest **הרגולטורית** (תקנות 2017) היא לרוב הדרישה המחמירה ביותר שחלה בפועל, מחמירה יותר מהדרישה הכללית (הלא-מספרית) של ISO/IEC 27001 עצמו. יש לוודא שה-SoA (פרק 2, 6.1.3) וה-Risk Treatment Plan משקפים את התדירות הרגולטורית המחייבת בפועל, לא רק "תדירות סבירה" כללית.

הפלט של `score` ו-`report` (סעיפים 6.3, 9.6) כולל אף הוא את שדה `test_protocol` לכל ממצא — כך שבפועל אפשר לעבור ישירות מרשימת סיכונים מנוקדת (מה החומרה) לפרוטוקול הבדיקה שמאמת אותה (איך בודקים/מוודאים) מאותה הרצה בודדת של הכלי.

לחזרה לתוכן העניינים המלא — ראו [README.md](#). המשך ישיר: **פרק 10 — פרוטוקולי בדיקה מפורטים לפי CWE**.

פרק 10: פרוטוקולי בדיקה מפורטים לפי CWE

פרק 9 (סעיף 9.14) הציג טבלת "פרוטוקול בדיקה" תמציתית לכל אחד מ-41 ה-CWE-ים בכלי הנלווה — שורה אחת לכל חולשה, בדיוק כפי שמופיעה בקוד (`cwe_risk_calculator.py`). הפרק הזה **מרחיב כל שורה לפרוטוקול בדיקה מלא, שלב-אחר-שלב**: מטרה, תנאים מוקדמים, רצף פעולות, קריטריון הצלחה (מה מבדיל ממצא אמיתי מ-`false positive`), וכלים לדוגמה.

חשוב: כל הפרוטוקולים בפרק זה מיועדים **אך ורק** לביצוע במסגרת בדיקת חדירה מורשית, תחת ROE כתוב (פרק 9, סעיף 9.7) — על מערכת שיש לכם הרשאה מפורשת ובכתב לבדוק. חלקם (בעיקר הבדיקות מבוססות-פאזינג בקבוצה ד') דורשים סביבת בדיקה מבוקרת ואינם מתאימים להרצה מול פרודקשן חי כלל. אין בפרק זה קוד ניצול (`exploit`) פעיל — רק מתודולוגיית **אימות** קיום חולשה.

קבוצה א' — הזרקות ועיבוד קלט (Web / קלט משתמש)

CWE-79 — Cross-Site Scripting (XSS)

- **מטרה:** לאמת שקלט לא-מסונן מבוצע כקוד JavaScript בדפדפן קורבן.
- **תנאים מוקדמים:** מיפוי כל נקודות הקלט שמוחזרות/מוצגות בפלט HTML (פרמטרי URL, טפסים, כותרות, שמות קבצים שמוצגים חזרה למשתמש).
- **שלבים:** 1. הזרקת מחרוזת-ניקוד ייחודית (`canary`, למשל `xsscany123`) בכל נקודת קלט ומעקב היכן היא חוזרת בפלט. 2. עבור כל מיקום שהתגלה — בחירת `payload` מתאים להקשר: תגית HTML רגילה (`<script>alert(document.domain)</script>`), הקשר `attribute` (`<img src=x onerror=alert(1)>`), או הקשר `JS string`. 3. אימות **הרצה בפועל** בדפדפן (לא רק הופעת המחרוזת כטקסט בתגובה). 4. עבור חשד ל-XSS `stored`: שמירת ה-`payload` ואימות הפעלתו גם מסשן/משתמש אחר שצופה באותו תוכן מאוחר יותר. 5. עבור `DOM-based` XSS: בדיקת `sinks` ב-JavaScript בצד הלקוח (`document.write` , `innerHTML`) שמקבלים קלט מ-URL/hash ללא סניטציה.
- **קריטריון הצלחה:** קוד מבוצע בפועל (למשל חלון `alert` נפתח, בקשת `callback` ל-`listener` חיצוני מתקבלת) — לא הופעת תגית כטקסט בלבד.
- **כלים לדוגמה:** Burp Suite/OWASP ZAP (Proxy + Scanner), שירות `callback` ייעודי לזיהוי XSS עיוור (`blind XSS`).

CWE-89 — SQL Injection

- **מטרה:** לאמת שקלט משתמש מגיע ללא סניטציה לשאילתת SQL.
- **תנאים מוקדמים:** זיהוי כל פרמטר שסביר שמזון לשאילתה (פרמטרי חיפוש, מיון, סינון, מזהי רשומה).

- **שלבים:** 1. **בדיקה בוליאנית:** השוואת תגובה בין `id=1` לבין `id=1 AND 1=1` מול `id=1`.
 2. **בדיקה מבוססת-שגיאה:** הזנת תו בודד (') ובדיקת הבדל בתגובה מעיד על הזרקה.
 3. **בדיקה מבוססת-זמן** (עיוורת, ללא הבדל תגובה נראה לעין): הזנת הופעת שגיאת DB בתגובה.
 4. **אימות מבוקר:** רק לאחר אישור סבירות גבוהה ומדידת זמן תגובה. `-- SELECT SLEEP(5);`
 משלבים 1-3, ואך ורק תחת ROE שמתיר זאת — ניסיון חילוץ מוגבל (UNION-based) של פרט לא-רגיש (למשל גרסת DB) כדי להוכיח את ההיקף, **ללא** חילוץ מידע רגיש בפועל.
- **קריטריון הצלחה:** שינוי מבוקר ומדיד בהתנהגות/בתוכן התגובה שתלוי ישירות במבנה השאילתה שהוזרק, לא רק שגיאה כללית חד-פעמית.
- **כלים לדוגמה:** Burp Suite (Intruder/Repeater ידני), sqlmap (רק לאימות/תיעוד תחת ROE מפורש — לא כברירת מחדל).

CWE-78 — OS Command Injection

- **מטרה:** לאמת שקלט משתמש מגיע ללא סניטציה לקריאת מערכת הפעלה (shell).
- **שלבים:** 1. זיהוי פרמטרים שסביר שמשמשים בקריאה חיצונית (שם קובץ, כתובת IP ל-ping, פרמטר "המרה"/"עיבוד" וכו').
 2. הזנת מפריד פקודות (; , pipe, && , `\$( )`) ואחריו פקודה שקטה כמו `sleep 5`.
 3. מדידת זמן תגובה מול קריאה זהה ללא הזרקה — עיכוב עקבי מעיד על ביצוע הפקודה.
 4. אימות משני: פקודה שמפיקה פלט נצפה (למשל `whoami`) רק אם ROE מתיר ניצול פעיל ולא רק אימות עיוור.
- **קריטריון הצלחה:** עיכוב/פלט תלוי-פקודה עקבי וניתן לשחזור, לא רעש רשת מקרי.
- **כלים לדוגמה:** Burp Suite Repeater, תיעוד ידני של זמני תגובה.

CWE-94 — Code Injection (כולל SSTI)

- **מטרה:** לאמת שקלט משתמש מתפרש כקוד בשפת התבנית/סקריפט של השרת.
- **שלבים:** 1. הזנת ביטוי חשבוני ניטרלי בהקשר תבנית (`{{7*7}}` , `{7*7}` , `#{7*7}`) בהתאם למנוע התבנית (החשוד).
 2. אם התגובה מציגה 49 במקום המחרוזת המקורית — סימן לביצוע בפועל (Server-Side Template Injection).
 3. זיהוי מנוע התבנית הספציפי (`/jinja2/Twig` / `FreeMarker` וכו') לפי תחביר שגיאה או תגובה לביטויים ייחודיים למנוע.
 4. תיעוד ההיקף מבלי לבצע קריאת קוד שרירותי בפועל, אלא אם ROE מתיר.
- **קריטריון הצלחה:** תוצאה מחושבת (49) ולא הביטוי הגולמי מוחזרת כטקסט.
- **כלים לדוגמה:** רשימת payloads ייעודית ל-SSTI לפי מנוע (ידני/Burp).

CWE-22 — Path Traversal

- **מטרה:** לאמת גישה לקבצים מחוץ לתיקיית התוכן המיועדת.
- **שלבים:** 1. זיהוי פרמטר שמייצג שם/נתיב קובץ (הורדת קובץ, טעינת תבנית, לוגו).
 2. הזנת רצפי `../` (מקודדים ולא-מקודדים) לקובץ ידוע ביעד (`/etc/passwd` / בלינוקס, `/web.config` / `boot.ini` בווידוס).
 3. אם מסונן — ניסיון עקיפות נפוצות: קידוד כפול (`%252e%252e%252f`), null-byte (`%00`), נתיב מוחלט ישיר.
 4. אימות תוכן הקובץ שהוחזר תואם לקובץ הידוע (לא רק קוד 200 ריק).

- **קריטריון הצלחה:** תוכן קובץ אמיתי מחוץ להיקף המיועד מוחזר בתגובה.
- **כלים לדוגמה:** Burp Intruder עם רשימת payloads טרורסל מוכנה.

CWE-434 — Unrestricted Upload of Dangerous File Type

- **מטרה:** לאמת אפשרות להעלות קובץ מבוצע (webshell) ולהריץ אותו בשרת.
- **שלבים:** 1. ניסיון העלאת קובץ עם סיומת מבוצעת (.php , .jsp , .aspx) בהתאם לפלטפורמת השרת שזוהתה. 2. אם נחסם לפי סיומת — ניסיון double extension (shell.php.jpg , null-byte (shell.php%00.jpg) , או שינוי Content-Type/magic bytes. 3. איתור נתיב הקובץ שהועלה (לרוב חשוף בתגובת ה-API/בדפדוף תיקיות). 4. גישה ישירה לקובץ שהועלה ואימות שהוא **מתבצע** בשרת (לא רק נשמר כקובץ סטטי).
- **קריטריון הצלחה:** קוד מהקובץ שהועלה מתבצע בפועל בצד השרת בתגובה לבקשת GET/POST ישירה אליו.
- **כלים לדוגמה:** Burp Suite, webshell מינימלי לבדיקה (<?php echo "canary123"; ?>) — לא (webshell פעיל מלא) לצורך אימות בלבד.
- **המשך חקירה:** אם התגלה קובץ חשוד אמיתי בסביבה הנבדקת (לא רק ה-PoC שלכם) — לפני שמריצים/פותרים אותו, סרקו אותו סטטית עם כלי מוכר כמו YARA + חוקי PHP-Malware-Finder או Semgrep (ראו [tools/README.md](#)) לפני כל בדיקה ידנית נוספת.

CWE-502 — Deserialization of Untrusted Data

- **מטרה:** לאמת שנקודת קצה מקבלת ומפענחת (deserialize) אובייקטים מסודרים ממקור לא מהימן.
- **שלבים:** 1. זיהוי תעבורה שמכילה פורמט סריאליזציה מוכר (Java: r00 , PHP: "...":0:8 , Python pickle: opcode headers). 2. אימות שהשרת אכן מפענח (לא רק מעביר) את המידע — למשל שינוי ערך בשדה מספרי בתוך האובייקט המסודר ובדיקת תגובה שונה. 3. אך ורק תחת ROE מפורש: שימוש ב-gadget chain ידוע וכלי ייעודי (ysoserial למשל) כדי להוכיח ביצוע קוד מבוקר (POC בלבד, לא payload הרסני).
- **קריטריון הצלחה:** אישור שהשרת מבצע לוגיקה שמקורה באובייקט שנשלט על ידי הבודק (למשל callback חוץ-לפס לשרת נשלט).
- **כלים לדוגמה:** (PHP) phpggc , (Java) ysoserial — במסגרת ROE בלבד.

CWE-611 — XML External Entity (XXE)

- **מטרה:** לאמת שמנתח XML פותר ישויות חיצוניות (external entities).
- **שלבים:** 1. איתור נקודות קצה שמקבלות XML (כולל פורמטים "נסתרים" כמו SOAP, קבצי Office/SVG שמפוענחים בשרת). 2. שליחת מסמך עם <!ENTITY xxe ...> , בוגף המסמך. 3. אם אין תגובה ישירה בוגף התשובה — בדיקת exfiltration חוץ-לפס (OOB): ישות חיצונית שמפנה ל-URL/DNS נשלט על ידי הבודק.

- **קריטריון הצלחה:** תוכן קובץ מקומי מוחזר בתגובה, או קריאת רשת יוצאת נצפית לשרת ה-OOB בשליטת הבודק.
- **כלים לדוגמה:** Burp Suite (Collaborator לבדיקת OOB).

CWE-601 — Open Redirect

- **מטרה:** לאמת שפרמטר הפניה מוביל לדומיין חיצוני שרירותי.
- **שלבים:** 1. איתור פרמטרים מסוג redirect/return URL (next=, return_url=, ?). 2. הזנת דומיין חיצוני נשלט (https://attacker-controlled.example) (redirect=) ומעקב אחר קוד תגובה א3/כותרת Location. 3. בדיקת עקיפות נפוצות של allow-list (נתיב יחסי מטעה, //, @ בין דומיינים).
- **קריטריון הצלחה:** הדפדפן/הלקוח מנותב בפועל לדומיין החיצוני שהוזן.
- **כלים לדוגמה:** Burp Repeater, בדיקה ידנית בדפדפן.

CWE-20 — Improper Input Validation (כללי)

- **מטרה:** לאתר שדות שאין להם ולידציה עקבית, כבסיס לחולשות ממוקדות יותר.
- **שלבים:** 1. מיפוי כל שדות הקלט (טפסים, פרמטרי API, כותרות HTTP). 2. הזנה שיטתית לכל שדה: ערכים מסוג שגוי (מחרוזת בשדה מספרי), אורך קיצוני, תווים מיוחדים/יוניקוד, ערכים שליליים/אפס. 3. תיעוד כל תגובה חריגה (שגיאה לא-מטופלת, קריסה, שינוי התנהגות) לצורך חקירה ממוקדת בהמשך (הזרקה/עקיפת ולידציה עסקית).
- **קריטריון הצלחה:** זיהוי שדה שמקבל קלט שלא היה אמור לעבור ולידציה בסיסית — משמש כמפת דרכים לשלבי בדיקה ממוקדים יותר.
- **כלים לדוגמה:** Burp Intruder עם רשימות fuzzing סטנדרטיות.

קבוצה ב' — זהות, אימות והרשאות

CWE-352 — Cross-Site Request Forgery (CSRF)

- **מטרה:** לאמת שפעולה משנה-מצב מתבצעת ללא אימות מקור הבקשה.
- **שלבים:** 1. איתור פעולה רגישה (שינוי סיסמה/מייל, העברת כספים, שינוי הרשאות) ובדיקה אם היא דורשת טוקן 2. CSRF. בניית דף HTML חיצוני עם טופס/בקשת fetch שמבצעת את אותה פעולה, ללא הטוקן (או עם טוקן שגוי/ישן). 3. הרצת הדף בדפדפן בזמן שמתמש-בדיקה מחובר ("קורבן"), ואימות שהפעולה בוצעה בפועל בחשבון.
- **קריטריון הצלחה:** הפעולה מבוצעת בהצלחה בחשבון הקורבן ללא ידיעתו, רק מכוח ביקורו בעמוד חיצוני.
- **כלים לדוגמה:** HTML PoC פשוט + דפדפן, Burp Suite (CSRF PoC generator).

CWE-287 — Improper Authentication

- **מטרה:** לאמת חולשה במנגנון האימות עצמו (לא בהרשאות שלאחריו).

- **שלבים:** 1. מיפוי כל נתיבי האימות (login רגיל, SSO, API keys, "זכור אותי", שחזור סיסמה). 2. בדיקת עקיפת 2FA (למשל דילוג ישיר לשלב שלאחר האימות השני, שינוי ערך `mfa_verified` בתשובת ה-API/עוגיה). 3. בדיקת session fixation: קביעת session ID לפני login ובדיקה אם הוא נשאר תקף לאחריו. 4. ניסיון brute-force מבוקר (קצב נמוך, לפי ROE) לזיהוי חוסר rate limiting.
- **קריטריון הצלחה:** מעקף אימות מוצלח (גישה ללא סיסמה תקפה/ללא גורם שני), או session שלא מתחדש בהתחברות.
- **כלים לדוגמה:** Burp Suite (Sequencer לבדיקת session tokens), Intruder לבדיקת rate limiting.

CWE-306 — Missing Authentication for Critical Function

- **מטרה:** לאמת שפעולה קריטית נגישה ללא כל אימות.
- **שלבים:** 1. מיפוי endpoints מנהליים/קריטיים (לרוב מגילוי בקוד JS צד-לקוח, תיעוד API, או ניחוש נתיבים נפוצים כמו `/admin`, `/api/internal`). 2. שליחת בקשה ישירה ללא כל כותרת אימות (ללא cookie/Authorization). 3. השוואת התגובה מול קריאה זהה עם אימות תקף — תגובה זהה (לא קוד 401/403) מעידה על חולשה.
- **קריטריון הצלחה:** הפעולה הרגישה מתבצעת/המידע מוחזר ללא כל אימות.
- **כלים לדוגמה:** Burp Repeater, סריקת נתיבים (dirsearch/ffuf) לאיתור endpoints לא-מתועדים.

CWE-862 — Missing Authorization

- **מטרה:** לאמת שפעולה שאמורה לדרוש הרשאה ספציפית לא באמת בודקת אותה.
- **שלבים:** 1. יצירת שני משתמשי-בדיקה: אחד ברמת הרשאה גבוהה, אחד רגיל (או ללא session כלל). 2. שליחת אותה בקשה מדויקת עם ה-session של המשתמש הרגיל/ללא session. 3. השוואת תוכן התגובה מול המשתמש המורשה — כפי שנעשה בפועל בדוגמת pentest-milatova (פרק 9.6): שדות context-edit הושוו בין תגובה מאומתת ללא-מאומתת.
- **קריטריון הצלחה:** מידע/פעולה שאמורים להיות מוגבלים מוחזרים/מתבצעים גם למשתמש שאינו אמור להיות מורשה.
- **כלים לדוגמה:** Burp Repeater עם שני פרופילי session מוגדרים מראש.

CWE-863 — Incorrect Authorization (IDOR)

- **מטרה:** לאמת שניתן לגשת לנתוני/משאבי משתמש אחר על ידי שינוי מזהה.
- **שלבים:** 1. יצירת שני חשבונות בדיקה (A ו-B) עם נתונים מובחנים בכל אחד. 2. איתור בקשה שמכילה מזהה אובייקט (`?id=`, `/users/123/`) בחשבון A. שינוי המזהה למזהה השייך לחשבון B, תוך שימוש ב-session של A בלבד. 4. אימות אם נתוני B מוחזרים/ניתנים לשינוי דרך session של A.
- **קריטריון הצלחה:** גישה/שינוי מוצלחים לנתוני משתמש B באמצעות הרשאות משתמש A בלבד.
- **כלים לדוגמה:** Burp Repeater/Autorize extension (בדיקת הרשאות אוטומטית בין תפקידים).

CWE-269 — Improper Privilege Management

- **מטרה:** לאמת אפשרות להסלים הרשאות מעבר למותר.
- **שלבים:** 1. פענוח/בדיקת מבנה טוקן ההרשאה (JWT claims, פרמטר role בבקשה). 2. שינוי שדה role/permission (למשל "role": "user" ל- "role": "admin") ושליחה מחדש. 3. בדיקה אם השרת מאמת את הערך מול מקור פנימי (DB/session) או מקבל ללא ביקורת את מה שהלקוח שלח.
- **קריטריון הצלחה:** פעולה שדורשת הרשאה גבוהה יותר מתבצעת בהצלחה לאחר שינוי הערך בצד הלקוח בלבד.
- **כלים לדוגמה:** jwt_tool לפענוח/מניפולציית JWT, Burp Repeater.

CWE-284 — Improper Access Control (כללי)

- **מטרה:** כיסוי שיטתי-רוחבי מעבר לבדיקות הממוקדות למעלה.
- **שלבים:** 1. בניית מטריצת endpoint × role/פעולה עבור כל התפקידים בהיקף הבדיקה. 2. בדיקת כל תא במטריצה (אוטומטית ככל האפשר) מול ההתנהגות בפועל. 3. ריכוז חריגות שנמצאו לכדי ממצא אחד מסכם, במקום דיווח נקודתי בלבד.
- **קריטריון הצלחה:** זיהוי שיטתי, לא מקרי, של כל שילובי role/פעולה החורגים מהמדיניות המתועדת.
- **כלים לדוגמה:** Burp Authorize, גיליון מטריצה ידני לתיעוד.

קבוצה ג' — ניהול סודות

CWE-798 / CWE-259 — Hard-coded Credentials / Password

- **מטרה:** לאתר סודות (API keys, סיסמאות) קבועים בקוד/בתצורה.
- **שלבים:** 1. חיפוש מחרוזות חשודות בקוד המקור (grep אחר password=, api_key=, secret=, מפתחות בפורמט מוכר כמו AKIA ל-2. AWS). סריקת היסטוריית git (כולל commits שנמחקו) לאיתור סודות שהוסרו מאוחר יותר אך נשארו בהיסטוריה. 3. בדיקת קבצי תצורה חשופים (.env, config.json, גיבויים) שנגישים דרך שרת ה-4. Web. עבור אפליקציות מובייל: פירוק (decompile) ה-APK/IPA וחיפוש סודות שהוטמעו בבינארי. 5. אימות שהסוד שנמצא **פעיל בפועל** (למשל מפתח API אכן מתקבל על ידי השירות) — לא רק שנראה כמו סוד.
- **קריטריון הצלחה:** סוד שנמצא מתקבל בפועל על ידי מערכת אמיתית (לא רק "נראה" כמו credential).
- **כלים לדוגמה:** trufflehog/gitLeaks (סריקת היסטוריית git), jadx/apktool, ripgrep/grep (פירוק אפליקציות אנדרואיד).

CWE-321 — Hard-coded Cryptographic Key

- **מטרה:** לאתר מפתחות הצפנה/חתימה קבועים.

- **שלבים:** זהה במהותו ל-CWE-798, בדגש על חיפוש תבניות מפתח (PEM headers, מחרוזות Base64 באורך אופייני למפתח) בקוד/בינארי, ואימות שהמפתח שנמצא משמש בפועל לחתימה/פענוח בסביבת הפרודקשן (למשל ניסיון חתימת טוקן תקף עם המפתח שנמצא).
- **קריטריון הצלחה:** יצירת טוקן/חתימה תקפה עם המפתח שאותר, המתקבלת על ידי המערכת כאמיתית.
- **כלים לדוגמה:** jwt_tool, סקריפט חתימה ידני לפי אלגוריתם שזוהה.

CWE-522 — Insufficiently Protected Credentials

- **מטרה:** לאמת ששידור/אחסון סיסמאות אינו חשוף.
- **שלבים:** 1. בדיקה אם סיסמה/טוקן מועברים כפרמטר ב-URL (מופיע בלוגים/היסטוריה). 2. בדיקת דגלי עוגיה (Secure, HttpOnly) על עוגיות ששן/אימות. 3. אם יש גישה לבסיס הנתונים (ROE מרשה) — בדיקת אלגוריתם ה-hashing (bcrypt/argon2 מול MD5/SHA1 ללא salt).
- **קריטריון הצלחה:** סיסמה/טוקן נחשפים בטקסט גלוי בערוץ לא-מוגן, או hashing חלש מאומת ישירות מול הנתונים.
- **כלים לדוגמה:** Burp Suite (בדיקת בקשות/עוגיות), hashcat לזיהוי סוג hash (ללא crack בפועל, אלא אם ROE מתיר).

קבוצה ד' — חשיפת מידע

CWE-200 — Exposure of Sensitive Information

- **מטרה:** לאמת חשיפת מידע שאמור להיות מוגבל למשתמשים מסוימים בלבד.
- **שלבים:** 1. שליחת אותה בקשה כמאומת (הרשאה מלאה) וכלא-מאומת/מורשה-חלקית. 2. השוואת diff מדויק בין שתי התגובות (שדות, אורך, מבנה JSON). 3. סימון כל שדה שמופיע רק בתגובה ה"אמורה" להיות מוגבלת יותר, אך מופיע בפועל גם בתגובה הפחות-מורשית.
- **קריטריון הצלחה:** שדה בעל רגישות (PII, מזהה פנימי, דגל הרשאה) נחשף בהקשר שלא אמור לקבל אותו.
- **כלים לדוגמה:** Burp Comparer (diff בין תגובות).

CWE-209 — Sensitive Info in Error Messages

- **מטרה:** לאמת חשיפת פרטי תשתית דרך הודעות שגיאה.
- **שלבים:** 1. שליחת קלט לא-תקין במגוון סוגים (טיפוס שגוי, JSON פגום, ערך חורג) לכל endpoint. 2. בדיקת תגובת השגיאה: stack trace, נתיב קובץ מקומי, גרסת framework/ספרייה, שם משתמש DB. 3. תיעוד כל endpoint שמחזיר מידע מעבר להודעת שגיאה גנרית.
- **קריטריון הצלחה:** פרט תשתיתי קונקרטי (נתיב, גרסה, שם פנימי) נחשף בתגובת שגיאה שהמשתמש הקצה יכול לגרום לה.
- **כלים לדוגמה:** Burp Intruder עם רשימת קלטים "שוברי-פרסינג".

CWE-732 – Incorrect Permission Assignment

- **מטרה:** לאתר משאבים עם הרשאות רחבות מדי.
- **שלבים:** בדיקת הרשאות קבצים בשרת (אם יש גישה), ובענן – בדיקת policy של דליים/buckets (public read/write), הרשאות IAM רחבות מדי, שיתוף קבצים ציבורי לא-מכוון.
- **קריטריון הצלחה:** משאב נגיש/ניתן-לשינוי על ידי גורם שלא אמור להיות מורשה, מאומת בבקשה ישירה (לא רק סריקת policy).
- **כלים לדוגמה:** CloudFox/ScoutSuite (סקירת תצורת ענן), `aws s3 ls` ציבורי לבדיקת buckets חשופים.

CWE-668 – Exposure of Resource to Wrong Sphere (Multi-tenant)

- **מטרה:** לאמת בידוד תקין בין דיירים (tenants) במערכת SaaS.
- **שלבים:** יצירת שני חשבונות-דייר נפרדים, וניסיון גישה למשאבי דייר B דרך session/API key של דייר A (דומה במהותו ל-IDOR ברמת הדייר כולו, לא רק ברמת רשומה בודדת).
- **קריטריון הצלחה:** גישה מוצלחת למשאב של דייר אחר.
- **כלים לדוגמה:** Burp Repeater עם שני חשבונות-דייר מוגדרים.

CWE-1188 – Insecure Default Initialization

- **מטרה:** לאתר רכיבים שנשארו בתצורת ברירת המחדל של היצרן.
- **שלבים:** זיהוי רכיבים (DB, לוח בקרה, שירות ענן) מסוג ידוע, וניסיון פרטי גישה סטנדרטיים (admin/admin, ברירת מחדל מתועדת של היצרן), ובדיקת פאנלים חשופים לאינטרנט ללא הגנה.
- **קריטריון הצלחה:** כניסה מוצלחת עם פרטי ברירת מחדל, או גישה לפאנל ניהולי ללא כל אימות.
- **כלים לדוגמה:** Shodan/censys לאיתור פאנלים חשופים (סביבת recon), רשימות פרטי ברירת מחדל ידועות.

CWE-16 – Security Misconfiguration (כללי)

- **מטרה:** סקירת תצורה כללית כשלב recon מוקדם.
- **שלבים:** בדיקת headers אבטחה (CSP, HSTS, X-Content-Type-Options), קבצי חשיפה (`.env`, `.git/`, `robots.txt`), גיבויים בנתיב ציבורי, שירותים חשופים שאינם נחוצים (פורטי ניהול, ממשקי דיבוג).
- **קריטריון הצלחה:** תצורה חריגה קונקרטית שמאפשרת גישה/מידע מעבר לצפוי (למשל `.git/` מלא נגיש בהיסטוריית קוד מלאה).
- **כלים לדוגמה:** `tools/fingerprint.py` ב-`pentest-booknet` (ראו פרק 9.6), Nikto, שכבת recon בסיסית של Burp.

CWE-1021 — Clickjacking

- **מטרה:** לאמת שניתן להטמיע את הדף ב-iframe זדוני.
- **שלבים:** בדיקת headers (Content-Security-Policy: frame- , X-Frame-Options , ancestors), ובניית עמוד HTML בדיקה שמטמיע את היעד ב- <iframe> ומאמת שהוא נטען (לא נחסם על ידי הדפדפן).
- **קריטריון הצלחה:** הדף נטען בפועל בתוך ה-iframe החיצוני.
- **כלים לדוגמה:** עמוד HTML מינימלי + דפדפן.

קבוצה ו' — לוגיקה וזיכרון (רכיבי native)

בקבוצה זו הבדיקה היא **פאזינג ותצפית**, לא בניית קוד ניצול — הפרק אינו כולל הדרכת exploitation למעבר לזיהוי ותיעוד ההתרסקות/ההתנהגות החריגה.

CWE-190 — Integer Overflow or Wraparound

- **שלבים:** הזנת ערכים בקצוות טווח (0, שלילי, MAX_INT , MAX_INT+1) בשדות מספריים המשפיעים על הקצאת משאבים/מכסות, ובדיקת עקיפת בדיקת תקציב או הקצאת זיכרון שגויה כתוצאה מכך.
- **קריטריון הצלחה:** התנהגות שגויה נצפית ונשחזרת (עקיפת מכסה, שגיאת הקצאה) בעקבות ערך קצה ספציפי.

CWE-125 / CWE-787 — Out-of-bounds Read / Write

- **שלבים:** פאזינג (fuzzing) קלט בינארי/פרוטוקול מותאם עם ערכי אורך/אינדקס חריגים, בסביבת בדיקה מבודדת עם instrumentation (sanitizers כמו ASAN) להאצת גילוי קריסות; תיעוד כל קריסה עם קלט משחזר (test case) ו-stack trace.
- **קריטריון הצלחה:** קריסה נשחזרת עם קלט קונקרטי, עם אינדיקציה (מ-sanitizer) לגישת זיכרון מחוץ לתחום.

CWE-416 — Use After Free

- **שלבים:** בדיקה דינמית עם sanitizers בסביבת פיתוח/בדיקה בלבד, ממוקדת ברצפי קריאה שמשחררים ואז ניגשים שוב למבנה נתונים — לא בדיקה ישירה מול פרודקשן.
- **קריטריון הצלחה:** אינדיקציית sanitizer מפורשת ל-use-after-free עם קלט משחזר.

CWE-476 — NULL Pointer Dereference

- **שלבים:** שליחת קלטים חסרים/ריקים (שדות JSON חסרים, מערכים ריקים) לכל נקודת קצה, ותצפית על קריסה/שגיאת שרת פנימית (5xx) חוזרת ונשנית.
- **קריטריון הצלחה:** קריסה/DoS נשחזרים בעקביות עם קלט חסר ספציפי.

CWE-311 / CWE-319 – Missing Encryption / Cleartext Transmission

- **שלבים:** ניתוח תעבורת רשת מלאה (mitmproxy/Wireshark) בין לקוח לשרת ובין שירותים פנימיים; איתור HTTP לא-מוצפן, פרוטוקולים legacy (FTP/Telnet), ובדיקת אחסון (גיבויים, buckets) ללא הצפנה at-rest.
- **קריטריון הצלחה:** מידע רגיש (סיסמה, טוקן, PII) נצפה בפועל בתעבורה גולמית בלכידת חבילות.

CWE-327 – Broken or Risky Cryptographic Algorithm

- **שלבים:** הרצת סריקת (testssl.sh/sslyze) TLS לאיתור פרוטוקולים/ cipher suites חלשים; בדיקת קוד/תיעוד API לאלגוריתמי hashing/הצפנה ישנים (MD5, SHA1, DES, RC4).
- **קריטריון הצלחה:** אלגוריתם חלש מאומת בפועל בשימוש (לא רק תמיכה תיאורטית ב-cipher suite).

CWE-330 – Insufficiently Random Values

- **שלבים:** דגימת סדרת טוקנים/מזהי-session/קישורי reset-password (לפחות כמה עשרות דגימות), ניתוח סטטיסטי לדפוס/entropy נמוך (למשל ערכים עוקבים או תלויי-זמן), וניסיון לחזות/לשחזר טוקן עתידי מהדפוס.
- **קריטריון הצלחה:** טוקן עתידי/קיים משוחזר בהצלחה מתוך ניתוח הדפוס.

CWE-295 – Improper Certificate Validation

- **שלבים:** הצבת MITM proxy מבוקר בין הלקוח (אפליקציה/שירות-לשירות) לשרת עם תעודה לא-תקינה (self-signed/פגת-תוקף/שם לא-תואם), ובדיקה אם הלקוח מקבל את החיבור ללא שגיאה.
- **קריטריון הצלחה:** הלקוח משלים את הפעולה (login, סנכרון נתונים) מול ה-proxy למרות תעודה פסולה.

CWE-346 – Origin Validation Error (CORS)

- **שלבים:** שליחת בקשה עם כותרת Origin שרירותית ובדיקת התגובה (Access-Control-Allow-Origin ו-Allow-Credentials); בדיקה ספציפית אם * משולב בטעות עם credentials, או אם ה-Origin שנגשל משתקף (reflected) ללא רשימת היתר אמיתית.
- **קריטריון הצלחה:** קריאה חוצה-מקור עם עוגיות/טוקן של הקורבן מצליחה מדומיין שרירותי שאינו ברשימת ההיתר המיועדת.

CWE-918 — Server-Side Request Forgery (SSRF)

- **שלבים:** 1. איתור פרמטר שגורם לשרת לבצע בקשת HTTP חיצונית (webhook, "בדיקת URL", ייבוא תמונה/קובץ מ-URL). 2. הזנת כתובת מטא-דאטה פנימית של ענן (169.254.169.254) או כתובת פנימית (127.0.0.1, טווח רשת פנימי). 3. אם אין תגובה ישירה — בדיקת SSRF עיוור באמצעות callback ל-listener חיצוני בשליטת הבודק (אישור שהשרת אכן ביצע את הבקשה היוצאת).
- **קריטריון הצלחה:** תוכן ממשאב פנימי מוחזר, או קריאת רשת יוצאת מהשרת נצפית ב-listener החיצוני.
- **כלים לדוגמה:** Burp Collaborator, שרת HTTP פשוט לניטור callbacks.

CWE-400 — Uncontrolled Resource Consumption

- **שלבים:** **אך ורק** בסביבת בדיקה מבוקרת ובאישור ROE מפורש — שליחת קלט/עומס חריג (payload גדול, ביטוי regex עם נטייה ל-ReDoS, ריבוי בקשות מבוקר) ומדידת זמן תגובה/צריכת משאבים מול ROE שמגדיר סף מוסכם.
- **קריטריון הצלחה:** האטה/כשל שירות משוחזרים בעקביות עם קלט/עומס ספציפי, בתוך גבולות ROE (ולא הרצת DoS ממשי ללא אישור).

המשך: פרק 11 — כיצד מתגלים חולשות Zero-Day. לחזרה לתוכן העניינים — ראו [README.md](#).

פרק 11: כיצד מתגלים חולשות Zero-Day

מיקוד הפרק: זהו פרק **מודעות וניהול סיכונים**, לא מדריך טכני לכתיבת קוד ניצול (exploit). המטרה: להבין את **המתודולוגיה** שבה חוקרי אבטחה מגלים חולשות לא-ידועות, כדי שארגון יוכל לקבל החלטות ניהול סיכונים נכונות (הגנת-עומק, מדיניות גילוי אחראי, ציפיות ריאליות ממערך ה-Threat Intelligence שלו) — לא כדי לפתח יכולת תקיפה. מחקר חולשות אמיתי במערכות של מישהו אחר, ללא הרשאה מפורשת (בעלות על המערכת, תוכנית bug bounty מוגדרת, או מחקר על קוד פתוח/מוצר בבעלותכם), הוא לרוב עברה פלילית — ראו סעיף 11.6.

11.1 מהו Zero-Day, ולמה זה שונה מ"חולשה רגילה"

Zero-Day (יום אפס) הוא חולשה שאין לה עדיין תיקון (patch) זמין — המונח מתאר את מספר הימים שהיצרן ידע עליה לפני שתוקן: אפס. ברגע שמתפרסם תיקון, החולשה הופכת ל-N-day (חולשה מתועדת עם CVE, שהתיקון עברה זמן אך לא כל הארגונים התקינו אותו עדיין — ובפועל, רוב הניצול האמיתי בשטח הוא של N-days, לא של Zero-Days אמיתיים).

ההבדל המהותי לניהול סיכונים: מול חולשה ידועה (N-day) יש **בקרה ברורה וזמינה** — A.8.8 (ניהול חולשות טכניות, פרק 3) אומר בעצם "התקינו את התיקון בזמן סביר". מול Zero-Day אין תיקון להתקין — הבקרה היחידה האפשרית היא **הגנת-עומק** (Defense in Depth): בקורות שמצמצמות נזק גם כשהחולשה הספציפית לא ידועה מראש (סעיף 11.5).

11.2 מי בפועל מחפש Zero-Days, ולמה

קבוצה	מניע	ערוץ פרסום/שימוש
חוקרי אבטחה עצמאיים / חוקרי bug bounty	תגמול כספי, מוניטין מקצועי	דיווח לספק/פלטפורמת bug bounty (HackerOne, Bugcrowd)
צוותי אבטחה פנימיים של יצרנים (למשל Google Project Zero)	חיזוק המוצר/האקוסיסטם	דיווח פנימי + פרסום אחרי תיקון (Coordinated Disclosure)
חברות מחקר אבטחה מסחריות	לקוחות ארגוניים/ממשלתיים	דוחות מחקר, לעיתים מכירת גישה בשוק אפור/שוק ממשלתי
גורמי מדינה / APT	ריגול, פעולות סייבר התקפיות	שימוש חשאי, לרוב לא מדווח כלל עד לגילוי בשטח
פושעי סייבר	רווח כספי (כופרה, גניבת מידע)	שימוש/מכירה בפורומים פליליים

הפרק הזה עוסק **בטכניקות המחקר עצמן** (שזהות בבסיסן בין הקבוצה הלגיטימית ללא-לגיטימית — ההבדל הוא **מה עושים עם הממצא**, לא איך מוצאים אותו), ומתמקד ביישום האחראי: חוקר עצמאי/צוות פנימי שמדווח באחריות.

11.3 שיטות מחקר עיקריות (ברמה מושגית)

Fuzzing (פאזינג)

הזנת קלט אקראי/מוטציה-מבוסס (mutation-based) או מובנה-פורמט (generation-based) לתוכנה, תוך ניטור קריסות/התנהגות חריגה. **Coverage-guided fuzzing** (כלים כמו ++AFL, libFuzzer) משתמש ב-instrumentation כדי להנחות את המוטציות לכיוון נתיבי קוד שטרם נבדקו – משמעותית יעיל יותר מפאזינג "טיפש" (dumb fuzzing) אקראי לחלוטין. זו השיטה שהניבה את החלק הארי מהחולשות שדווחו בעשור האחרון ברכיבי native (מפענחי קבצים, ספריות פרסינג, פרוטוקולי רשת).

סקירת קוד (Code Review / Static Analysis)

כשקוד המקור זמין (קוד פתוח, גישה כחלק מ-bug bounty, white-box pentest – פרק 9.2), חוקרים סורקים ידנית או בעזרת כלים אוטומטיים (Semgrep, CodeQL) אחר תבניות חולשה מוכרות (שרשראות קלט-לפלט לא-מסוננות, שימוש מסוכן בפונקציות ידועות). זו השיטה האפקטיבית ביותר לחולשות לוגיות (broken access control, race conditions) שפאזינג נוטה לפספס כי הן לא גורמות לקריסה.

הנדסה לאחור (Reverse Engineering)

כשקוד המקור לא זמין (תוכנה קניינית, קושחה/firmware, אפליקציות מובייל/מכשירים), חוקרים משתמשים ב-disassemblers/decompilers (כמו IDA Pro, Ghidra) כדי לשחזר הבנה של הלוגיקה הפנימית ולזהות בה חולשות – לרוב בשילוב עם פאזינג ממוקד (fuzzing מונחה-ידע על המבנה הפנימי).

Patch Diffing (ניתוח הבדלי תיקון)

לאחר שספק מפרסם עדכון אבטחה **מבלי לפרט** מה בדיוק תוקן (נפוץ מאוד), חוקרים משווים (diff) את הקוד/הבינארי לפני ואחרי התיקון כדי להבין מה תוקן – ומכאן להסיק את החולשה המקורית. זו הדרך המרכזית שבה **N-day** הופך לניצול מהיר: הפער בין פרסום תיקון (שחושף את קיום הבעיה) לבין שהארגון מתקין אותו הוא חלון ניצול ריאלי וקצר בהרבה מרוב הארגונים מניחים.

ביצוע סימבולי / Concolic Testing

טכניקות מתקדמות יותר (כלים כמו KLEE, angr) שמנתחות את כל הנתיבים האפשריים בקוד באופן מתמטי-פורמלי, ולא רק מריצות קלטים בפועל – יעילות לאיתור חולשות עמוקות בתוכנה קריטית (למשל קושחת מכשירים רפואיים, פרוטוקולי אבטחה) אך דורשות משאבי חישוב ומומחיות גבוהים.

11.4 הגילוי האחראי (Coordinated Vulnerability Disclosure)

זה בדיוק המקום שבו שני התקנים שהוזכרו בפרק 9 (סעיף 9.15) — **ISO/IEC 29147** (Vulnerability disclosure) ו-**ISO/IEC 30111** (Vulnerability handling processes) — נכנסים לתמונה:

1. **החוקר מדווח** לספק בערוץ מוגדר (security.txt בכתובת `/.well-known/security.txt`, תוכנית bug bounty, כתובת מייל ייעודית) — לפי העקרונות ב-ISO/IEC 29147.
2. **הספק מאשר קבלה**, מנתח ומאמת את הדיווח, ומפתח תיקון — לפי התהליך הפנימי המתואר ב-ISO/IEC 30111.
3. **תקופת embargo מוסכמת** (נהוג 90 יום, מוסכמה שהפכה כמעט תקן דה-פקטו בזכות Google Project Zero, אך משתנה לפי חומרה/מורכבות) — החוקר נמנע מפרסום עד שהתיקון מוכן, בתמורה להתחייבות הספק לתקן בפרק זמן סביר.
4. **פרסום מתואם** (Coordinated Disclosure) — לאחר שהתיקון זמין, שני הצדדים מפרסמים (לרוב CVE + כתיבה טכנית של החוקר).
5. **חלופת קיצוץ**: אם הספק לא מגיב/מתעלם לאורך זמן סביר, חלק מהחוקרים עוברים ל-**Full Disclosure** (פרסום ללא תיאום) — שנוי במחלוקת מקצועית, ומשמש בעיקר כמנוף לחץ כשהערוץ המתואם נכשל.

תוכניות Bug Bounty (HackerOne, Bugcrowd, ותוכניות פרטיות של יצרנים גדולים) הן הערוץ המוסדי המרכזי כיום לגילוי אחראי בתמורה לתגמול — ומהוות גם דרך לגיטימית לארגון עצמו "לגייס" מחקר חולשות על המוצר שלו (ראו 11.5).

11.5 מה זה אומר לארגון (לא לחוקר) במונחי ISO/IEC 27001

מכיוון שאי אפשר "לתקן" חולשה לא-ידועה מראש, ניהול הסיכון ל-Zero-Day מתמקד בשלוש חזיתות משלימות:

1. **הגנת עומק (Defense in Depth)** — בקורות שמפחיתות נזק **גם כשחולשה ספציפית לא ידועה**: הרשאות מינימליות (least privilege, A.5.15/5.18), הפרדת רשתות (A.8.22), ניטור התנהגותי במקום מבוסס-חתימה בלבד (EDR שמזהה פעילות חריגה, לא רק מזהה קובץ זדוני ידוע — A.8.16), ו-WAF עם virtual patching (חסימת דפוס ניצול גם בלי לדעת את שם ה-CVE).
2. **מודיעין איומים (A.5.7, פרק 3)** — מעקב אחרי אינדיקציות לניצול Zero-Day "בשטח" (in the wild) לפני שיצא תיקון רשמי — לרוב מגיע דרך יצרני אבטחה, CERT לאומיים, או קהילות שיתוף מידע ענפיות (ISACs).
3. **מהירות תגובה לכשהחולשה הופכת ל-N-day** — ברגע שמתפרסם CVE/תיקון, מרוץ הזמן מתחיל (Patch Diffing, 11.3) — כאן A.8.8 (ניהול חולשות טכניות) הוא הבקרה הקריטית: **מהירות** ההטמעה של תיקון קובעת אם הארגון נחשף לחלון הניצול שנפתח ברגע פרסום ה-patch, לפני שתוקפים מספיקים להנדס-לאחור את התיקון בעצמם (Patch Diffing, סעיף 11.3).

4. **תוכנית גילוי אחראי/bug bounty עצמאית** — ארגון עם מוצר/שירות משלו יכול "לגייס" קהילת חוקרים (11.4) לגלות חולשות **לפני** שגורם עוין יגלה אותן — בפועל מרחיב את משאבי ה-A.8.29 (בדיקות אבטחה) מעבר לצוות הפנימי/ספק ה-pentest היחיד.

המסקנה: תקן ISO/IEC 27001 עצמו לא "פותר" את בעיית ה-Zero-Day (אי אפשר לדרוש מארגון "לגלות מראש את הבלתי-ידוע") — אבל בקורותיו (הגנת-עומק, ניהול חולשות, מודיעין איומים, בדיקות אבטחה) הן בדיקת המנגנון שמצמצם את חלון הפגיעות **מסביב** לבלתי-ידוע, ולא רק מול הידוע.

11.6 גבולות אתיים ומשפטיים — קריטי מכל מה שנאמר בפרק זה

- **מחקר חולשות על מערכת שאינה שלכם, ללא הרשאה מפורשת (בעלות, ROE חתום, או תוכנית bug bounty מוגדרת בהיקפה), הוא לרוב עברה פלילית** ברוב שיטות המשפט — בישראל, בין היתר תחת חוק המחשבים, התשנ"ה-1995. זהה בדיקת לעיקרון ה-ROE שנדון לעומק בפרק 9 (סעיף 9.7) — הוא חל באותה מידה על מחקר Zero-Day כמו על pentest מוזמן.
- **תוכניות bug bounty מגדירות היקף (scope) מפורש** — מחקר מחוץ להיקף המוגדר (למשל תשתית פנימית שלא נכללה, או ניסיון גישה לנתוני משתמשים אמיתיים מעבר לנדרש להוכחת החולשה) חורג מההרשאה גם כשיש תוכנית bug bounty פעילה.
- **Full Disclosure ללא תיאום** (11.4) עלול לחשוף משתמשי-קצה לסיכון מידי לפני שקיים תיקון — שיקול דעת מקצועי ואתי, לא רק טכני.
- פרק זה **אינו** מהווה הדרכה לפיתוח קוד ניצול (exploit development) בפועל — למי שמתעניין בכך במסגרת לגיטימית (מחקר אקדמי, red team מורשה, תוכנית bug bounty), הנהיב המקובל הוא הכשרה מובנית (OSCE3/OSEP, פרק 9.15) ותרגול בסביבות מעבדה ייעודיות (HackTheBox, CTF, מוסדר), לא ניסוי-וטעייה על מערכות ייצור.

לחזרה לתוכן העניינים המלא — ראו [README.md](#). זהו הפרק האחרון בספר.

כלים נלווים לספר

cwe_risk_calculator.py – מחשבון סקר סיכונים לפי CWE

כלי שורת-פקודה המפעיל אופליין לחלוטין (ללא רשת, ללא תלות בחבילות חיצוניות) המיישם מודל ניקוד סיכונים **Likelihood x Impact** (מטריצת 5x5), בהתאם למתודולוגיה המתוארת ב-ISO/IEC 27005 ומיושמת בסעיף 6.1.2 של ISO/IEC 27001 (ראו פרק 2 ו-פרק 6).

הכלי כולל טבלת בסיס קטנה ומתוחזקת-ידנית של חולשות **CWE** נפוצות (מבוססת בעיקרה על חולשות שחוזרות ברשימות "CWE Top 25 Most Dangerous Software Weaknesses"), עם הצעות ברירת-מחדל לסבירות ולהשפעה על סודיות/שלמות/זמינות לכל CWE. אלו **ערכי פתיחה להמחשה בלבד** – כל ארגון חייב לכייל אותם למציאות שלו (סוג הנכס, חשיפה, בקורות מפצות קיימות וכו') לפני שהם משמשים בפועל בהצהרת ההתאמה (SoA) או ברשומת הסיכונים הרשמית.

שימוש

```
# הבסיסית הכלולה בכלי CWE-הצגת טבלת ה
python3 cwe_risk_calculator.py list

# בודד, עם דריסת ברירת המחדל של הסבירות CWE ניקוד
python3 cwe_risk_calculator.py score --cwe CWE-89 --asset "פורטל לקוחות" --likelihood 3

# והפקת רשומת סיכונים ממוינת (CSV או JSON) ניקוד קובץ ממצאים שלם
python3 cwe_risk_calculator.py report --input sample_findings.json --output risk_register.csv

# ים-CWE-ספציפי, או עבור כל 41 ה CWE עבור (test protocol) פרוטוקול בדיקת חדירה
python3 cwe_risk_calculator.py protocol --cwe CWE-862
python3 cwe_risk_calculator.py protocol
```

כל רשומת CWE בטבלת הבסיס כוללת גם שדה `test_protocol` – תיאור קונקרטי של טכניקת הבדיקה שבדק חדירה מורשה מפעיל כדי לאמת את קיום החולשה בפועל (לא רק ניקוד חומרה תיאורטי). השדה הזה מופיע גם בפלט `report / score`, כך שממצא מנוקד ופרוטוקול האימות שלו יוצאים מאותה הרצה. ראו פרק 9, סעיף 9.14 בספר לטבלה המלאה (זהה לזו שבקוד) ולהסבר על הרעיון.

מבנה קובץ ממצאים (JSON או CSV) לפקודת `report`: שדה חובה `cwe_id`, ושדות אופציונליים `asset`, `likelihood`, `impact_c`, `impact_i`, `impact_a`, `notes` – כל שדה שלא סופק נלקח מטבלת ברירת המחדל של הכלי. דוגמה כללית ב- `sample_findings.json`, ודוגמה מבוססת על שני ממצאי **pentest** אמיתיים מתוך `pentest-milatova/` (בשורש הריפוי) ב- `pentest_case_study_findings.json` – ראו פירוט מלא בפרק 9 של הספר (9.6), כולל הסבר מדוע ממצא אחד דורש דריסה ידנית של ברירת המחדל כדי לשקף נכון את ההקשר.

מודל הניקוד

$$\text{score} = \text{likelihood} (1-5) \times \max(\text{impact_c}, \text{impact_i}, \text{impact_a}) (1-5)$$

טווח ציון	רמת סיכון
4-1	נמוך (Low)
9-5	בינוני (Medium)
16-10	גבוה (High)
25-17	קריטי (Critical)

מודל זה הוא דוגמה פדגוגית פשוטה ונפוצה, לא דרישה נורמטיבית של התקן — ISO/IEC 27005 משאיר את בחירת המתודולוגיה המדויקת לארגון, ובלבד שהיא עקבית וניתנת לשחזור (ראו סעיף 6.1.2 בפרק 2).

אנליטיקה ידנית של קוד PHP/Python/JS-TS חשוד — כלים קיימים ומוכרים

בשונה משאר הכלים בתיקייה זו, לצורך הסקירה הידנית של קובץ חשוד (למשל webshell שהתגלה בעקבות ממצא CWE-434, פרק 10) **אין צורך בכלי חדש** — זהו תחום שבו קיימים כלי קוד-פתוח בוגרים, נבדקים היטב, ומתוחזקים באופן פעיל. הטבלה הבאה ממפה כל שפה לכלי המקובל בתעשייה, עם פקודת התקנה והרצה מינימלית. כולם **סטטיים** (לא מריצים את הקובץ הנסרק) וניתנים להרצה אופליין לאחר ההתקנה החד-פעמית (חלקם דורשים הורדת חוקים/רשימות חתימה מעודכנות מדי פעם מהאינטרנט):

שפה/היקף	כלי	התקנה	הרצה בסיסית
רב-לשוני (PHP/Python/JS/TS ועוד)	Semgrep — מנוע כללי-בדיקה סטטיים עם ruleset ציבורי עצום	<code>pip install semgrep</code>	<code>semgrep --config p/security-audit --config p/secrets ./path</code>
PHP (webshells) malware (במיוחד)	-PHP-Malware Finder — חוקי YARA ייעודיים לזיהוי webshells וקוד PHP מעורפל	<code>git clone https://github.com/nbs-system/php-malware-finder</code> <code>pip install yara-python</code>	<code>yara -r php-malware-finder/php.yar suspect.php</code>
Python (SAST)	Bandit — סורק אבטחה סטטי רשמי של PyCQA	<code>pip install bandit</code>	<code>bandit -r ./path</code>
/JavaScript TypeScript	ESLint + eslint-plugin-security	<code>npm install --save-dev eslint</code>	<code>npx eslint --plugin security --rule '{"security/detect-</code>

שפה/היקף	כלי	התקנה	הרצה בסיסית
	eslint-plugin-no-unsanitized	eslint-plugin-security	eval-with-expression:"error"}' ./path
npm (שרשרת אספקה)	npm audit ידועות, ועיון ידני ב- scripts (/ preinstall postinstall) בקובץ package.json	מובנה ב-npm	npm audit
חתימות כלליות (כל שפה, כל סוג קובץ)	YARA עצמו, עם חוקים כלליים (כמו Neo23x0/signature-base)	pip install yara-python	yara -r rules.yar suspect_dir/

עיקרון מנחה זהה לשאר הספר: תוצאת כל אחד מהכלים האלה היא **עוגן לבדיקה אנושית**, לא פסק דין אוטומטי — Semgrep/Bandit/ESLint-security מייצרים גם False Positives בקוד לגיטימי (למשל שימוש תקין ב- eval בסקריפט build), ו-YARA תלוי לחלוטין באיכות ובעדכניות קובץ החוקים שנטען.

לניתוח סטטי עמוק ומבוסס-ML **ל-Python בלבד** (מיצוי תכונות מ-AST, מודל מאומן, הסבר בשפה טבעית — כלי פנימי של הריפוז הזה, לא כלי חיצוני), ראו את security_classifier/ בשורש הריפוז.

build_book_pdf.py — הפקת הספר כקובץ PDF יחיד

מרכיב את כל פרקי הספר (README.md + 11 - 01 + tools/README.md כנספח) לקובץ PDF אחד עם עימוד RTL תקין, תוכן עניינים מקושר (קישורים פנימיים בין הפרקים הופכים לעוגנים בתוך אותו PDF), טבלאות, ובלוקי קוד ב-LTR. דורש שתי תלויות אופציונליות שאינן חלק מהליבה של שאר הריפוז:

```
pip install weasyprint markdown

python3 iso27001-book/tools/build_book_pdf.py
# iso27001-book/ISO-27001-Summary-Book-HE.pdf : כותב ל
```

הריפוז מחדש בכל פעם שמעדכנים פרק, כדי שה-PDF המצורף יישאר מסונכרן עם קבצי ה-Markdown (מקור האמת של הספר).